



ALIENA REHAN

01-135222-031

NOOR TAHIR

01-135222-071

Not Safe For Work Content Detector

Bachelor of Science in Information Technology

Supervisor: Ms. Alyia Amir

Co-Supervisor: Kabeer Ahmed Bhatti

Department Of Computer Science,

Bahria University, Islamabad

October 2025

Abstract

With the rise of the internet age, millions of pictures and videos get uploaded every day on the internet by people around the globe. The unfortunate reality is that much of this content is unsuitable or harmful, including sexual content, violence, vulgar content, and child labor-related content. Such content is usually termed Not Safe For Work (NSFW) content. It is virtually impossible to manually screen such massive amounts of media; this is why we need to develop an automated system that can quickly and efficiently detect such content. This study presents an Automated NSFW Content Detection System, which is a web application used to upload images and videos and analyze their content to check their safety level. Such an automation process will be both simple and effective, thus proving useful for everybody including people as well as institutions. If a user wishes to analyze an image using the system, he simply needs to upload the image, which will then pass through various machine learning algorithms developed for identifying harmful content. This model is an example of OpenCLIP ViT-B-32 and applies what is known as zero-shot classification, meaning that it can classify any category of content without being specifically trained on each and every one of those categories. Other specialized models are additionally applied to increase accuracy when identifying categories of violent content, vulgar content, and child labor. Video content analysis, on the other hand, requires some modification in how the analysis is performed. As each video contains multiple frames, first, it is divided into several distinct frames using OpenCV, which is the computer vision library. Every frame is processed separately through the classification models. Thus, any potential harmful content will be identified in this way even if it occurs in one single frame.

The main part of the system uses PyTorch, which is a commonly used deep learning library that is responsible for inference/prediction of the results of the model. All the results of the analysis, such as predicted category of content and a level of certainty of the model represented by a score, are safely saved in a PostgreSQL database. In addition, all results will be available to the users for review at any time. Users receive information in the form of an easy-to-understand dashboard. There is the use of charts and graphs that allow users to easily comprehend the results and see what type of content was found and what is the level of confidence the model has regarding its prediction. Another benefit of this particular system is the fact that there is no scraping or crawling of social media sites for data collection purposes. Users have the option to submit the desired media files themselves, which provides more control over the system and makes it safer.

In summary, the project showcases that AI and deep learning can be used to develop an efficient content moderator application. With the use of automation to detect content that is not safe for work, the Internet becomes a much safer place for both individuals and corporations.

Acknowledgments

All gratitude is due to Almighty Allah, who gave us a little fraction of His vast wisdom, as well as for the good health and wellbeing required to finish this report. We wish to express our sincere thanks to Ms. Aliya Amir, Assistant Professor at the Department of Computer Science in Bahria University, for guiding us at various stages of the project and providing us with all the necessary facilities for the research. We are extremely thankful and indebted to her for sharing expertise, sincere and valuable guidance and encouragement. This project would not have been completed without her supervision, advice, and invaluable guidance. We are grateful to her for her support and encouragement throughout this project. We would want to take this opportunity to thank everyone of the department's faculty for their assistance and support. We also like to thank our parents for their constant love, care and support.

ALIENA REHAN and NOOR TAHIR

Islamabad, Pakistan

Contents

Abstract.....	i
Acknowledgments	ii
Acronyms and Abbreviations	viii
Chapter 1.....	1
Introduction	1
1.1. Background	1
1.2. Problem Description.....	2
1.3. Project Objectives.....	2
1.4. Project Scope	3
1.5. Application Areas	3
1.6. Organization of Report	3
Chapter 2.....	4
Literature Review	4
2.1 Comparative Study	4
2.2 Media Upload and Processing Module.....	6
2.2. Violence Detection	7
2.2.1. Weaponized Violence Detection	8
2.3. Vulgar Content Detection	8
2.4. Child Labor.....	9
2.5. Clean Content	9
Chapter 3.....	11
Requirements Specification	11
Existing System’s Specifications.....	11
Proposed System’s Specifications	11
3.1. Requirement Specifications.....	11
3.1.1. Software Requirements	11
3.1.2. Hardware Requirements	12
3.2. User Requirement Specifications	12
3.2.1. Functional Requirements.....	12

3.2.1.1.	Sign In.....	12
3.2.1.2.	Sign Up	12
3.2.1.3.	Media Upload	12
3.2.1.4.	Media Processing.....	13
3.2.1.5.	Analyzer.....	13
3.2.1.6.	Dashboard Display.....	13
3.3.	Non-Functional Requirements.....	13
3.3.1.	Internet Connection.....	13
3.3.2.	Performance	13
3.3.3.	Security	14
	User authentication is needed and Processed files are secure.....	14
3.3.4.	Scalability.....	14
3.3.5.	Availability.....	14
3.3.6.	Flexibility.....	14
3.3.7.	Reusability	14
3.4.	Assumptions and Dependencies	14
3.5.	Use Cases	14
3.5.1.	Sign Up Use Case.....	14
3.5.2.	Sign-In Use Case	15
3.5.3.	Logout Use Case.....	16
3.5.4.	Input Image Use Case.....	16
3.5.5.	Input video Use Case.....	17
3.5.6.	Analyzer Use Case	18
3.5.7.	Dashboard Use Case.....	19
Chapter 4.....		20
Design.....		20
4.1.	System Architecture.....	20
4.2.	Design Methodology	20
Low Level Design.....		22
4.3.	Use case Diagram.....	22
4.4.	Activity Diagrams	23

Database Design	32
4.6. Entity Relationship Diagram	32
4.7. GUI Design.....	32
Chapter 5.....	37
System Implementation	37
5.1. System Architecture.....	37
5.2. Processing Logic / Algorithms	39
Chapter 6.....	45
System Testing and Evaluation	45
6.1. Graphical User Interface (GUI) Testing	45
6.2. Usability Testing.....	45
6.2.1. Register Test Case	45
6.2.2. Login Test Case.....	47
6.2.3. Input Test Case	48
6.2.4. Processing Test Case	49
6.2.5. Analyzer Test Case.....	50
6.3. Software Performance Testing.....	51
6.4. Performance Testing	51
6.5. Evaluation Metrics for Deep Learning Models	52
6.5.1. Main Model.....	52
6.5.2. Violence model:	53
6.5.3. Vulgarity Model	54
6.6. Use of Additional Models (Improvement):	55
6.7. Compatibility Testing	56
6.8. Exception Handling.....	56
6.8.1. Register Test Case:	56
6.8.2. Login Test Case:.....	57
6.8.3. Input Test Case:	58
6.8.4. Model Processing Test Case:.....	58
6.8.5. Analyzer Test Case:.....	59
6.9. Load Testing	60
6.10. Security Testing	60

6.11.	Installation Testing	60
Chapter 7.....		61
7.1.	Conclusion.....	61
7.2.	Disciplined Project Management.....	61
7.3.	Team Communication	62
7.4.	Future improvements.....	62
Appendix A.....		63
User Manual		63
References		67

List of figures

Figure 1.1. Types Of Nsfw Content.....	2
Figure 2.1. Media upload	6
Figure 2.2. Types of classification.....	7
Figure 3.5.1. Sign Up Use Case	15
Figure 3.5.2. Sign Up Use Case	15
Figure 3.5.3. Log Out Use Case	16
Figure 3.5.4. Input Image Use Case	17
Figure 3.5.5. Input video Use Case	17
Figure 4.3.1. Use Case Diagram.....	22
Figure 4.4.1. Input Activity Diagram	23
Figure 4.4.2. Media Upload Activity Diagram.....	24
Figure 4.4.3. NSFW Analyzer Activity Diagram	25
Figure 4.4.4. Media Analyzer Activity Diagram	26
Figure 4.4.5. Display Activity Diagram	27
Figure 4.5.6. Login Sequence Diagram.....	28
Figure 4.5.7. Sign up sequence diagram.....	28
Figure 4.5.8 Media Uploaded Sequence Diagram.....	29
Figure 4.5.9. Media Processing Sequence Diagram.....	30
Figure 4.4.10. Analyzer Sequence Diagram.....	31
Figure 4.7.1. Sign Up Page.....	33
Figure 4.7.2. Log in Page	33
Figure 4.7.3. Dashboard	34
Figure 4.7.4 Dashboard	34
Figure 4.7.5. Analytics section 1	35
Figure 4.7.6. Analytics section 2	35
Figure 4.7.7. Subdivision of classifications.....	36
Figure 4.7.8. Graphical user interface (GUI)	36

Figure 5.1. High-level System Architecture	37
Figure 1- A. User Registration.....	63
Figure 2- A. User Login	64
Figure 3- A. Dashboard Overview.....	65
Figure 4- A. Media Upload.....	65
Figure 5- A. Output and Results	66

List of tables

Table 2.1. Comparative Study 1	4
Table 2.2. Comparative Study 2	5
Table 2.3. classification categories	8
Table 4. Sign Up Table	14
Table 5. Sign Up Table	15
Table 6. Log Out Table	16
Table 7. Input Image Table	16
Table 8. Input video Table	17
Table 9. Analyzer Table	18
Table 10. Dashboard Table	19
Table 11. System Internal Components	38
Table 12. Register Test Case.....	46
Table 13. Login Test Case	47
Table 14. Input Test Case	48
Table 15. Processing Test Case.....	49
Table 16. Analyzer Test Case	50
Table 17. Performance Testing	51
Table 18. Training Results (Main Model)	53
Table 19. Validation Results (Main Model).....	53
Table 20. Training Results (Violence Model).....	54
Table 21. Validation Results (Violence Model).....	54
Table 22. Training Results (Vulgarity Model).....	54
Table 23. Validation Results (Vulgarity Model).....	55
Table 24. Register Test Case.....	56
Table 25. Login Test Case	57
Table 26. Input Test Case	58
Table 27. Model Processing Test Case	59
Table 28. Analyzer Test Case	59

Acronyms and Abbreviations

AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
NSFW	Not Safe For Work
API	Application Programming Interface
UI	User Interface
DB	Database
SQL	Structured Query Language
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
GPU	Graphics Processing Unit
CPU	Central Processing Unit
NLP	Natural Language Processing
CLIP	Contrastive Language–Image Pretraining
OpenCLIP	Open-source implementation of CLIP
ViT	Vision Transformer
REST	Representational State Transfer

Chapter 1

Introduction

The purpose of this project is to build an advanced application for content analysis, which would enable automatic detection and classification of the three categories of harmful content – violence, vulgarity, and child labor – from images and videos uploaded by users. For content classification, the solution employs a pre-trained deep learning model as opposed to building one from scratch. OpenCLIP ViT-B-32 is used for zero-shot classification of images/videos through the method of comparing prompts. It enables increased flexibility and allows you to classify images/ videos without training a model. Classification output is shown in the form of a dashboard.

The primary function includes uploading media/content of your own from your device and running that content against machine learning algorithms for deeper analysis. Machine learning models will further categorize the videos/images into specific sub-categories, such as fighting, domestic violence, weaponized violence, nudity, partial nudity, and child labor content. There is a REST-based backend API for the frontend application that uploads media content and receives the classification output from there.

1.1. Background

Social media sites have been used as the primary means of sharing photos and videos among users of different age categories. Due to the increasing trend of producing and uploading digital content, photos and videos are shared in large quantities online. Although most of these visuals are harmless, some of them contain inappropriate and explicit content. This may include abusive, sexually explicit, or violent content that could have adverse effects on the viewers.

Violence involves any action causing or intending to cause harm to another individual. Violent actions might include physically assaulting another individual or using a weapon to harm someone. Along with violent imagery, one example of content requiring filtration from social media websites is NSFW (Not Safe For Work). This may involve nude or semi-nude images. Alternatively, SFW (Safe For Work) is content that poses no danger and is permissible for sharing on professional platforms.

As the number of media being uploaded increases, manual content monitoring becomes less efficient and unreliable. As such, content detection software becomes imperative. The principal aim of this project is to create a smart web-based tool that can detect NSFW media. This is different from other tools that depend on web crawling or scraping social media platforms. In this instance,

content analysis happens based on media files uploaded via the web interface. This project incorporates the use of an already trained deep learning model. OpenCLIP ViT-B-32 will facilitate zero-shot, prompt-driven classification of images and extracted video frames. Video frame extraction will be done through OpenCV and then classified using the developed classification pipeline. Some types of NSFW content mentioned in [Figure 1.1](#):

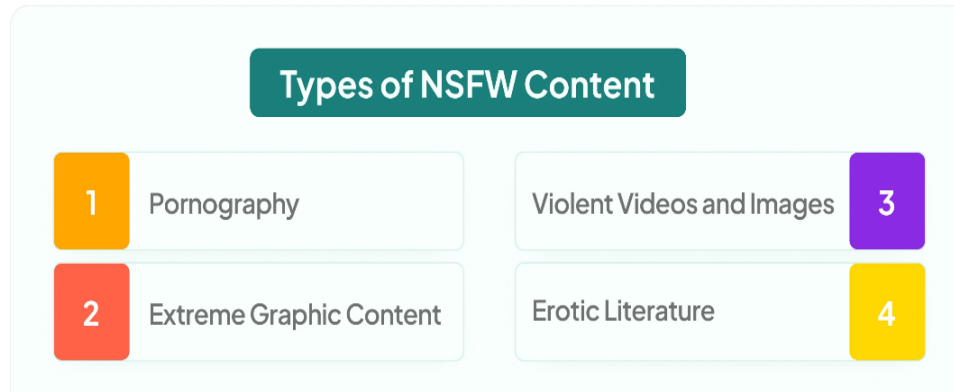


Figure 1.1. Types Of Nsfw Content

1.2. Problem Description

Existing content moderation algorithms rely heavily on large amounts of data with labels and re-training the model for each new category of content. This process is both resource-intensive and cumbersome. In this regard, the current project tries to solve this problem by employing a deep learning algorithm that has been pre-trained and is ready to conduct inference operations on any media files that users upload. There is a need for a service or functionality that will look at the content of the video or image and will perform some analysis to categorize the type of content within that image.

1.3. Project Objectives

- In order to design a system which detects NSFW content on a web interface where users can upload images/videos.
- To employ the technique of zero shot prompt-based classification using OpenCLIP ViT-B-32, which performs classification for images into four primary categories: Violence, Vulgarity, Child labor, and Clean.
- To extract frames from uploaded videos using OpenCV and pass them to classification process.
- To perform secondary subclassifications of violence as Physical violence, Weaponized violence, and Domestic violence.
- To also subclassify Vulgarity as Nudity and Partial Nudity.

- To show classification results along with confidence score on the dashboard in a graphical representation form.
- To store all the results in PostgreSQL database/local storage.
- To ensure our system does not depend on scraping/crawling of any external social media platform.

1.4. Project Scope

The key objective of this project is to develop an artificial intelligence-based model that can scan any image or video file uploaded to it to determine whether there is any offensive material like violence, obscene language, or child labor present. Our application can detect as well as identify violence, obscenity, and child labor within videos. With the help of this system, one can upload their image and videos and then use them for analyzing their contents. Our system will identify what kind of contents the user is uploading, and the result will be shown in the dashboard of the system.

1.5. Application Areas

- Educational organizations – to detect harmful content that gets posted in educational institutes.
- Organizations and workplaces – to make sure that only appropriate content appears on company websites.
- Government agencies – to help track down any harmful digital content.
- Content moderation organizations – to help humans in moderating content.
- Parental control software – to shield children from harmful content.

1.6. Organization of Report

This document is organized as follows. In the second Chapter [2](#) Literature view we will discuss the comparative study and similar work. Chapter [3](#) presents the requirement specifications while Chapter [4](#) details the system design. System implementation is discussed in Chapter [5](#) while Chapter [6](#) presents the results of the experimental evaluations. Finally, we conclude the report in Chapter [7](#).

Chapter 2

Literature Review

As a result of the growth in internet and social media usage, the users are now able to conveniently exchange digital content including images and videos. However, at the same time, there is an increase in the presence of indecent or inappropriate contents such as images of child workers, violence scenes, obscenity, and sexually explicit images. The term used to describe this content is NSFW (Not Safe For Work). Detecting and filtering such content is becoming more important in the field of computer vision and deep learning. A lot of research and technologies have been introduced to address this problem. This chapter highlights literature in areas like clean content classification, violence detection, child labor detection, vulgar content detection, and image/video upload and processing. Part of showing some problems in this work is through a comparative study of existing approaches.

2.1 Comparative Study

A variety of methods have been suggested by researchers and developers to identify and remove offensive material from digital communication channels. The following [Table 2.1](#) and [Table 2.2](#) summarize the related work reviewed for this project.

Table 2.1. Comparative Study 1

Ref	Name	Title	Journal /Book Year of Publication	Key Feature
[1]	Simonyan & Zisserman	Very Deep CNNs for Large-Scale Image Recognition (VGG)	2015	Used VGG16/VGG19 for image classification with deep CNN layers
[2]	Radford et al.	Learning Transferable Visual Models from Natural Language (CLIP)	2021	Zero-shot image classification using text-image matching

[3]	Howard et al.	Searching for MobileNetV3	2019	Lightweight CNN model for mobile/web-based image classification
[4]	Haider et al.	Detection of Violent Activities in Videos Using Deep Learning	2020	CNN-based violence detection in video frames using transfer learning

Table 2.2. Comparative Study 2

Ref	Name	Title	Journal /Book Year of Publication	Key Feature
[5]	Ali et al.	NSFW Content Detection Using Deep Neural Networks	2021	Multi-label NSFW classification using neural networks
[6]	ILO Report	Global Estimates of Child Labour	2020	Dataset and statistics on child labor used for model training
[7]	Yahoo Open NSFW	Open NSFW Model	2016	One of the first open-source models specifically for NSFW detection
[8]	Skin Sheriff Project	Skin-based Nudity Detection	2018	Skin detection technique for nudity classification achieving 91.9% accuracy

2.2 Media Upload and Processing Module

The system allows the users to provide a secure way for uploading their image/video files through a media upload system.

As soon as a media file gets uploaded by a user, it is processed by the backend server for the purpose of content detection. In case of image content, it is fed into the pipeline directly. However, in the case of video content, frames are extracted using OpenCV library and then are fed into the pipeline for classification purposes. The solution leverages pre-trained deep learning models to perform detection of NSFW. Zero-shot classification is implemented by leveraging OpenCLIP (ViT-B-32). The zero-shot classification leverages prompt-based detection where textual prompts like "nudity", "explicit content" or "safe content" are encoded and compared with visual embeddings extracted from the uploaded media file/image. The advantage of the prompt-based classification algorithm is that there is no need for further training of models.

The solution employs a pre-trained deep learning model to perform NSFW detection. Zero-shot classification is implemented by leveraging OpenCLIP (ViT-B-32). The zero-shot classification employs prompt-based detection where textual prompts like "violence", "vulgar content", "child labor" or "clean content" are encoded and compared with visual embeddings extracted from the uploaded media file/image. As with previous implementation, there is no need for further training of models. Inference is performed using the PyTorch framework.

The classification results together with confidence scores are shown on the system dashboard.



Figure 2.1. Media upload

2.2. Violence Detection

Much work has been done in the field regarding violence detection. One of the most significant advances in this field is that made by Haider et al., where they introduced a deep learning framework that identifies violent actions in videos through the use of CNNs. In this system, each video frame is examined and marked as either violent or non-violent. The algorithm used a massive database of violent and non-violent video snippets acquired from YouTube and gave outstanding performance on benchmark datasets. Nevertheless, the biggest drawback of their research was that their algorithm did not perform multi-classification; hence, it was only capable of identifying whether the video is violent or non-violent. In other words, their algorithm could recognize the occurrence of violence within a video but could not differentiate its type, which makes it challenging for institutions to respond appropriately. There were other researchers who studied the application of transfer learning models to enhance the accuracy of violence detection systems. Transfer learning is where a pre-trained model on a very large data set is adapted to perform a particular task. Some of these models include VGG16, VGG19, ResNet, and MobileNet. Trained on the massive ImageNet dataset containing millions of labeled images

There are some major subcategories of violence as given in figure, our major objective is to work on the following subclassifications of physical violence.



Figure 2.2. Types of classification

2.2.1. Weaponized Violence Detection

Weapon-related violence refers to the occurrence of imagery related to any weaponry including firearms, knives, and any weapon-like objects. Weapon-based violence recognition is essential as it is related to criminal activities as well as physical injury. Existing approaches employ CNN networks trained on a specific set of weapon-related images. Nevertheless, it requires considerable amount of data and cannot be used in unseen contexts. For this research project, we utilize the OpenCLIP framework instead of building a dedicated model. Weapon-based violence will be detected using image comparison based on text descriptions, such as "a person with a gun" or "knife attack". Using OpenCLIP framework that has learned on the enormous LAION-400M dataset, the system will be able to recognize weapon-based violence without requiring retraining. OpenCLIP is used in this project instead of training a dedicated model. It uses image comparison based on text prompts such as "person holding a gun" or "knife attack." As OpenCLIP is trained on a massive LAION-400M dataset, the system will be able to detect weapon-related violence without further training.

2.3. Vulgar Content Detection

The pictures or videos showing nudity and obscene materials can be called vulgar content. There have been several mechanisms to detect vulgar content, such as the Yahoo Open NSFW classifier; however, most classifiers classify images into two categories, namely, vulgar and non-vulgar. Some studies applied machine learning classifiers like Random Forest and SVM and proved to be accurate in their predictions. Nevertheless, Skin Sheriff had some difficulties due to its reliance on skin detection algorithms, as it could mistake beaches for vulgar content.

This project utilizes the hybrid mechanism of combining OpenCLIP with the other NSFW classifiers. It is noteworthy that the accuracy improved because of the use of OpenCLIP in dealing with edge cases via the prompt of text.

Table 2.3. classification categories

Classification Category	Description and Severity
Nudity	Depicts full exposure of sensitive private areas (e.g., genitals or female breasts). High severity.
Partial Nudity / Suggestive	Content that is highly suggestive, implying sexual activity, or featuring substantial but not complete exposure. Medium severity.
Graphic Sexual Content	Depicts explicit sexual acts (pornography). This is typically the highest severity class, regardless of whether it is real-life or animated.

The [table 2.3.](#) shows images The pictures or videos showing nudity and obscene materials can be called vulgar content. There have been several mechanisms to detect vulgar content, such as the Yahoo Open NSFW classifier; however, most classifiers classify images into two categories, namely, vulgar and non-vulgar.

Some studies applied machine learning classifiers like Random Forest and SVM and proved to

be accurate in their predictions. Nevertheless, Skin Sheriff had some difficulties due to its reliance on skin detection algorithms, as it could mistake beaches for vulgar content.

This project utilizes the hybrid mechanism of combining OpenCLIP with the other NSFW classifiers. It is noteworthy that the accuracy improved because of the use of OpenCLIP in dealing with edge cases via the prompt of text.

Table 2.3 shows the type of vulgarity that has been worked upon by other researchers and data scientists. The work that has been conducted in the field of detecting nudity and pornographic videos and images suggests that detecting explicitness in an image or video is not as difficult. Multiple different models were used to train the dataset which gave a maximum accuracy of 96.96% using Random Forest Algorithm and a minimum accuracy of 84.11% using Logistic Regression. These algorithms were used on AIIA-PID4 pornographic data set. Another project determined to detect pornography and other explicit images called the Skin Sheriff focused on detecting how much skin the person in the image is showing, based on that they would detect nudity and pornography. The project used a combination of datasets which included the Compaq Dataset, INRIA Holidays Dataset and images that were collected from different pornographic websites. The project achieved an accuracy of 91.9% using Linear SVM.

2.4. Child Labor

Child labor is a persistent global issue that affects both rich and poor countries equally. The use of children in any work that robs them of their youth, prevents them from attending regular school, and is intellectually, physically, socially, or morally detrimental is known as child labor. The International Labor Organization (ILO) estimates that there are over 215 million children worldwide who are engaged in child labor.

Since child labor is a sensitive topic, there has not been a lot of work done on this matter. Due to the sensitivity of the topic, there are no known datasets that are publicly available. Some data scientists in 2020 decided to build a model for detecting child labor through images. Since there are no known publicly available datasets, they created their own dataset from scratch using videos from YouTube. The dataset is based on the context of child labor occurring in Bangladesh and other south Asian countries such as Pakistan, Nepal, and India. Pre-trained models were used to achieve a maximum accuracy of 96.875% using pre-trained model and a minimum accuracy of 90.625% using Convolutional Neural Networks. Child labor detection still needs a lot more work to be done with a more versatile dataset.

2.5. Clean Content

By clean content, we refer to images or videos that do not have any violence, harmful or inappropriate content. Such images include images of daily activities performed by different individuals, informative images, nature images and others that follow all the required community guidelines. In our project, clean data is very important as not all data is harmful or inappropriate. Therefore, the system should be able to accurately detect clean data. The system will use deep learning algorithm to separate clean content from harmful content. In this case, the clean data acts as a standard class and thus makes it easy for the algorithm to detect harmful content. Failure by

the algorithm to distinguish clean data from other data will result into incorrect predictions. The correct prediction of clean data makes our system reliable and useful for our users. Some examples of clean data include images of nature, images of people performing normal tasks such as studying and others. However, a major shortcoming of our project is that it is not accurate in its prediction of clean data. For example, it classifies children studying or playing as Child Labor when they should be in clean.

Chapter 3

Requirements Specification

Existing System's Specifications

A number of approaches or tools can be employed for identifying the presence of violence or offensive content within any image/video. These approaches generally work independently as classification engines; these analyze an image or a video to determine its safety status and return a decision accordingly. Several existing approaches lack a comprehensive dashboard or visualization capabilities. Besides, there are certain platforms that depend on the process of retraining their models or integrate other platforms externally.

Proposed System's Specifications

The proposed system is a web-based NSFW content detection application that allows users to upload images or videos for automated analysis. Unlike systems that rely on social media scraping or crawling, this system directly processes media files uploaded by the user. The system uses a pre-trained deep learning model for classification. OpenCLIP (ViT-B-32) is used for zero-shot, prompt-based detection. The frames are collected using OpenCV, and they are analyzed individually. The frames received through the video inputs are analyzed separately with the use of OpenCV. The inference operation is performed with PyTorch. After the classification is done, the outputs are presented via a dashboard in graphical representations and statistics form. Graphical representation of the outputs can be used to view detection outcomes and their confidence scores. The solution concentrates on achieving fast, safe, and scalable NSFW detection.

3.1. Requirement Specifications

3.1.1. Software Requirements

The software requirements for this project are:

- Python (with PyTorch framework)
- OpenCLIP (ViT-B-32)
- OpenCV
- Browser Local Storage
- Backend Framework (Flask/RESTAPI)
- Frontend Framework (React, HTML, CSS, JavaScript)
- PyCharm (IDE)
- MS Word

- Overleaf

3.1.2. Hardware Requirements

The hardware requirements for this project are:

- Fast Internet Access
- Computer or Laptop with a dedicated GPU
- Computer or Laptop with at least 16GB of RAM

3.2. User Requirement Specifications

This is a secured web-based application which offers authentication, ability to upload media and dashboard for visualization purposes. It is important for users to register and then log into the system.

3.2.1. Functional Requirements

3.2.1.1. Sign In

- **Input:** To use this system, a user must enter his or her credentials (email address and password).
- **Output:** If the credentials provided are correct, the user is allowed to visit the dashboard of the website.
- **Process:** The system will check whether the credentials inputted are correct or not by checking the accounts created in the application.
- **Description:** The user needs to log in before he or she can use our web services. This is done by entering the correct credentials in order to use the web service.

3.2.1.2. Sign Up

- **Input:** The credentials required from the user are provided by him/her as asked from the sign-up module of the system.
- **Output:** Then the user will be redirected to the login page.
- **Process:** The account details will be saved by the system for future use.
- **Description:** To use the functionality provided by the system, it is mandatory that the user needs to go through the process of registration.

3.2.1.3. Media Upload

- **Input:** Image/Video.

- **Output:** File is processed for further analysis.
- **Process:** Media is kept temporarily on upload in the system.
- **Description:** The user will upload the media file for analysis of the same.

3.2.1.4. Media Processing

- **Input:** Uploaded image/video.
- **Output:** File is ready for analysis by the model.
- **Process:** If it is an image, it will pass directly to the model and if it is a video, then the video frames are extracted.
- **Description:** The uploaded file will be ready for analysis by the model.

3.2.1.5. Analyzer

- **Input:** Image or frame post-processing.
- **Output:** Classification output (either NSFW or Safe along with confidence).
- **Process:** Generation of visual embeddings from open clip, followed by zero shot classification using prompts and finally, executing inference with PyTorch.
- **Description:** The deep learning model classifies the content.

3.2.1.6. Dashboard Display

- **Input:** Classification outputs.
- **Output:** Visualization through graphs (bar chart, pie chart, statistics).
- **Process:** Outputs are displayed on the dashboard in form of graphs.
- **Description:** Analysis output display.

3.3. Non-Functional Requirements

3.3.1. Internet Connection

Response time and data processing are entirely dependent on the accessibility of the internet.

3.3.2. Performance

The application must be able to process uploaded images and analyze video frames in a timely manner. Better performance is achieved by using GPU support.

3.3.3. Security

User authentication is needed and Processed files are secure.

3.3.4. Scalability

Additional models and classification categories can be incorporated in the future.

3.3.5. Availability

As a web-based application, the system can be accessed anytime depending on server availability.

3.3.6. Flexibility

The application can use image and video files as input.

3.3.7. Reusability

The backend REST API can be used for other applications to analyze media contents.

3.4. Assumptions and Dependencies

The system assumes that uploaded media files are in supported formats. It also depends on pre-trained deep learning models for classification accuracy. No external social media APIs or web crawlers are used in this system.

3.5. Use Cases

3.5.1. Sign Up Use Case

This use case allows users to SIGN UP into the Database in order to get benefits from our system. As shown in [Table 4](#) and in [Figure 3.5.1](#):

Table 4. Sign Up Table

Use Case Name	Sign Up
Actor	User
Description	Enables a new user to create an account by providing an email address and password.
Pre-Condition	User is on the Sign-Up page and does not have an existing account with the provided email.
Post-Condition	A new user record is persisted in the database; User is redirected to the Login page.

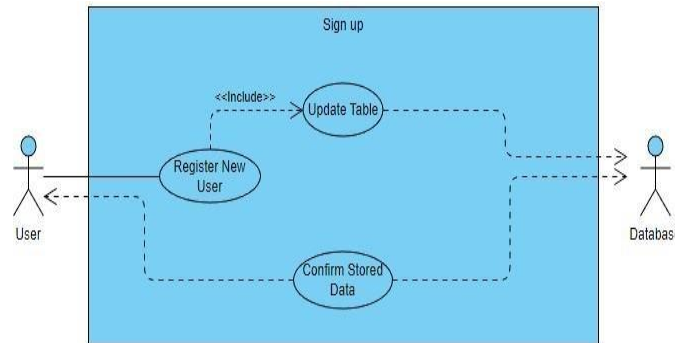


Figure 3.5.1. Sign Up Use Case

3.5.2. Sign-In Use Case

This use case allows the user to login into the system to access the relevant functions according to the user's role. To log in to the system, Users have to enter their unique id along with a password. Upon successful login, the user will be redirected to the next page. As shown in Table 5 and in Figure 3.5.2:

Table 5. Sign Up Table

Use Case Name	Sign In
Actor	User
Description	User logs into the system using registered credentials to access the dashboard.
Pre-Condition	User must be registered.
Post-Condition	User is redirected to dashboard upon successful authentication.

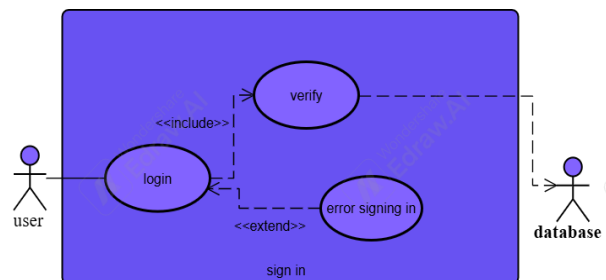


Figure 3.5.2. Sign Up Use Case

3.5.3. Logout Use Case

This Use Case is used to log out from our system. As shown in [Table 6](#) and in [Figure 3.5.3](#):

Table 6. Log Out Table

USE CASE NAME	Logout
Description	This use case allows users to log out of the system to secure their profile from unauthorized use.
Actor	User
Pre-Condition	User must be logged in first.
Post Condition	User will be displayed with login page.

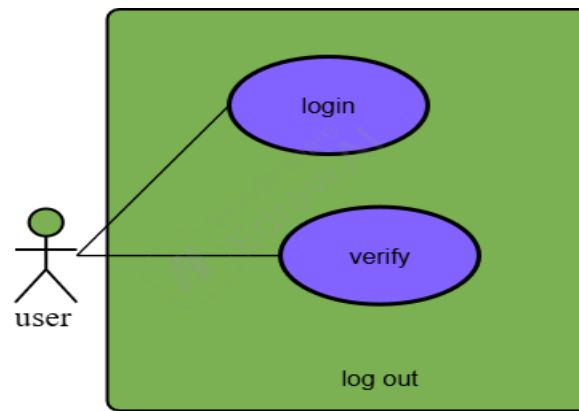


Figure 3.5.3. Log Out Use Case

3.5.4. Input Image Use Case

The user will enter an input after logging in; the user uploads an image file that the user wishes the system to analyze. As shown in [Table 7](#) and in [Figure 3.5.4](#):

Table 7. Input Image Table

Use Case Name	Upload Image
Actor	User
Description	After logging in, the user uploads an image file for NSFW detection and analysis.
Pre-Condition	User must be logged in.
Post-Condition	Image is sent to the analysis module and classification result is generated.

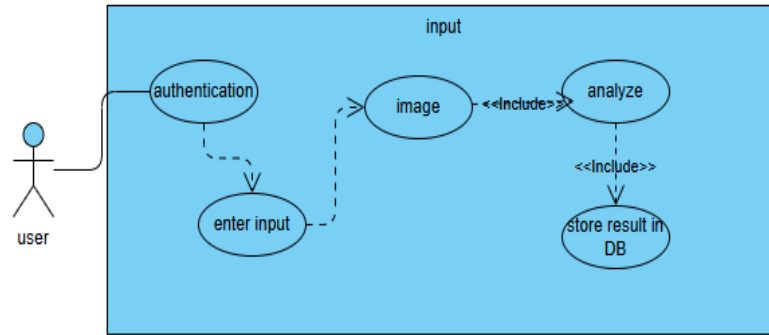


Figure 3.5.4. Input Image Use Case

3.5.5. Input video Use Case

This diagram shows the video upload workflow. After the user uploads a video, the system extracts frames using OpenCV. The frames are analyzed by pre-trained models, and the final result is stored, as shown in Table 8 and Figure 3.5.5:

Table 8. Input video Table

Use Case Name	Upload video
Actor	User
Description	After logging in, the user uploads an video and the system extracts frames using OpenCV for analysis.
Pre-Condition	User must be logged in.
Post-Condition	Video frames are analyzed and final classification result is generated.

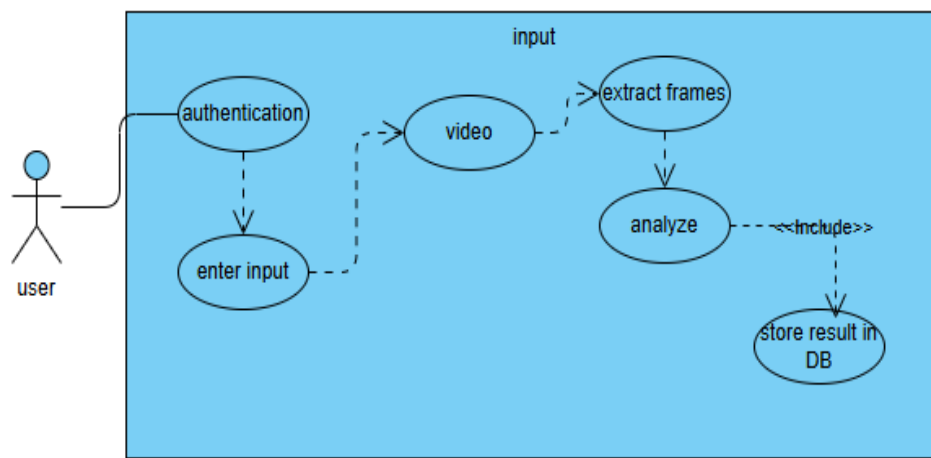


Figure 3.5.5. Input video Use Case

3.5.6. Analyzer Use Case

The user must press analyze button after the input has been entered. As shown in [Table 9](#) and [Figure 3.5.6](#).

Table 9. Analyzer Table

Use Case Name	Analyzer
Actor	System
Description	The system processes uploaded media using pre-trained models (OpenCLIP and other NSFW models) and performs classification using Torch inference.
Pre-Condition	Media file must be uploaded.
Post-Condition	NSFW/Safe result with confidence score is stored and displayed on dashboard.

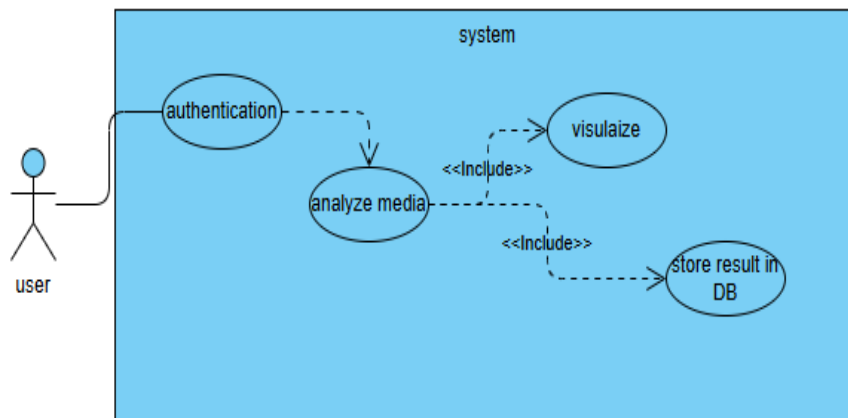


Figure 3.5.6. Analyzer Use Case

3.5.7. Dashboard Use Case

The user views stored classification results in graphical form. The system retrieves data and displays it as charts and statistics, as shown in Table 10 and Figure 3.5.7:

Table 10. Dashboard Table

Use Case Name	View Dashboard
Actor	User
Description	User views classification results in the form of charts and statistics.
Pre-Condition	User must be logged in and analysis must be completed.
Post-Condition	Dashboard displays graphical representation of results.

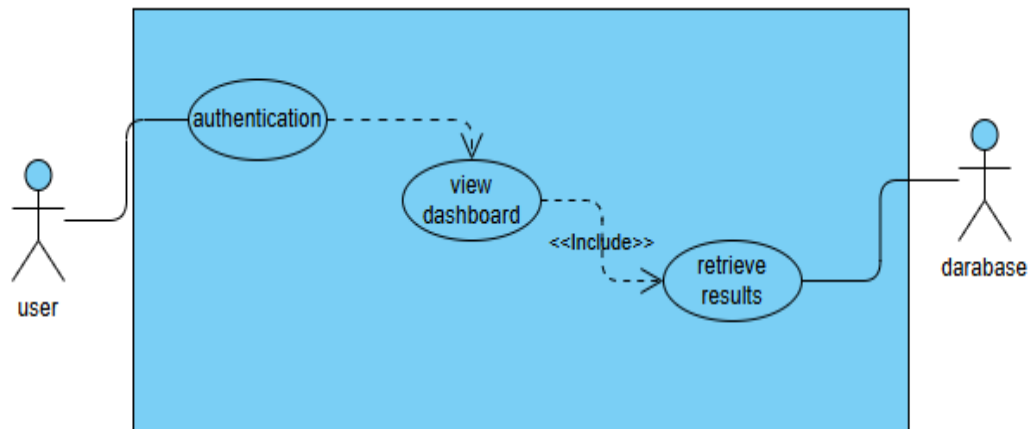


Figure 3.5.7. Dashboard Use Case

Chapter 4

Design

Systems design is the process of defining the architecture, components, modules, interfaces and data for a system to satisfy specified requirements.

The following sections constitute this chapter:

4.1. System Architecture

This section describes the system in narrative form using non-technical terms. It should provide a high-level system architecture diagram showing a subsystem breakout of the system, as shown in [Figure 4.1](#):

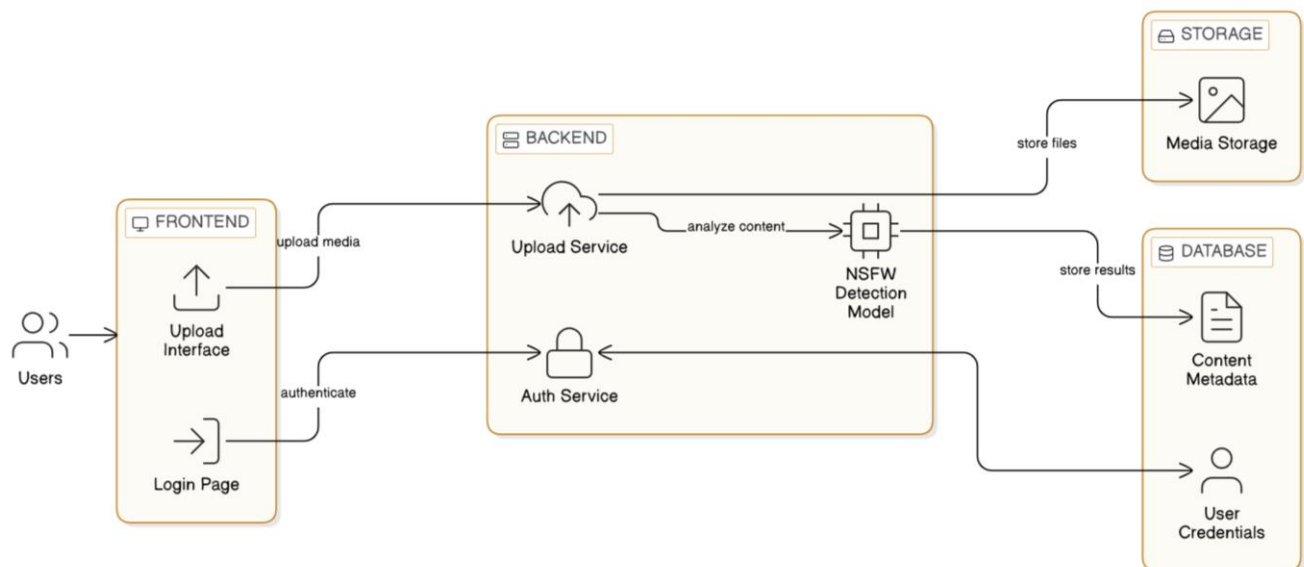


Figure 4.1. System Architecture

4.2. Design Methodology

The proposed project will be implemented using an agile methodology. When it comes to accomplishing this goal, the agile technique will prove very beneficial. According to the agile methodology, the planned project is divided into various phases. Each phase demands continuous

communication and feedback from team members, as well as continuous improvement. The agile model incorporates the following steps:

1. Planning
2. Requirements Analysis
3. Design
4. Development
5. Testing
6. Evaluation

The following [Figure 4.2.](#) shows the design methodology followed for the Dark Insight.

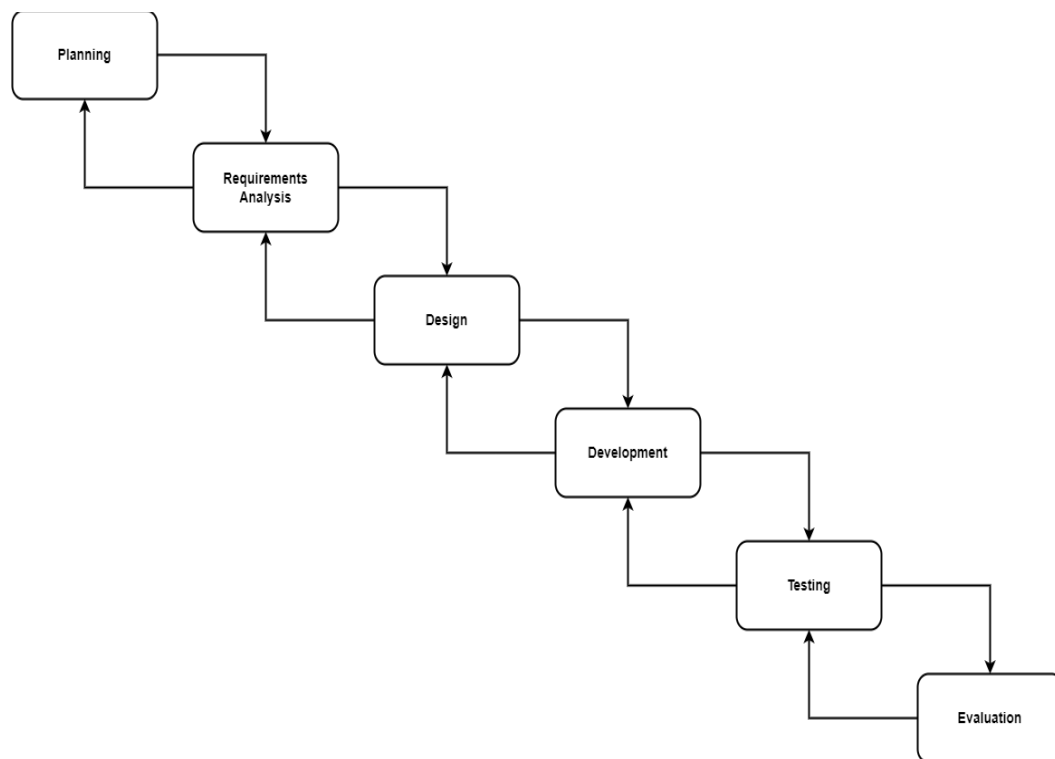


Figure 4.2. System Architecture

Low Level Design

4.3. Use case Diagram

4.3.1. Use case Diagram

The system's use case diagram shown in [Figure 4.3.1](#) illustrates how an actor interacts with the system. Initially, the user must log in to access all system features. After logging in, the user can upload an image or video file through the provided upload option. The system processes the uploaded media and performs NSFW (Not Safe For Work) content detection using a pre-trained zero-shot model. The analysis result determines whether the content is safe or contains inappropriate material. The detection results are then displayed to the user on the system dashboard. As shown in [Figure 3.5.6.](#), if the user wants to analyze a single image or video, they can simply upload the specific file and the system will immediately evaluate and present the classification outcome.

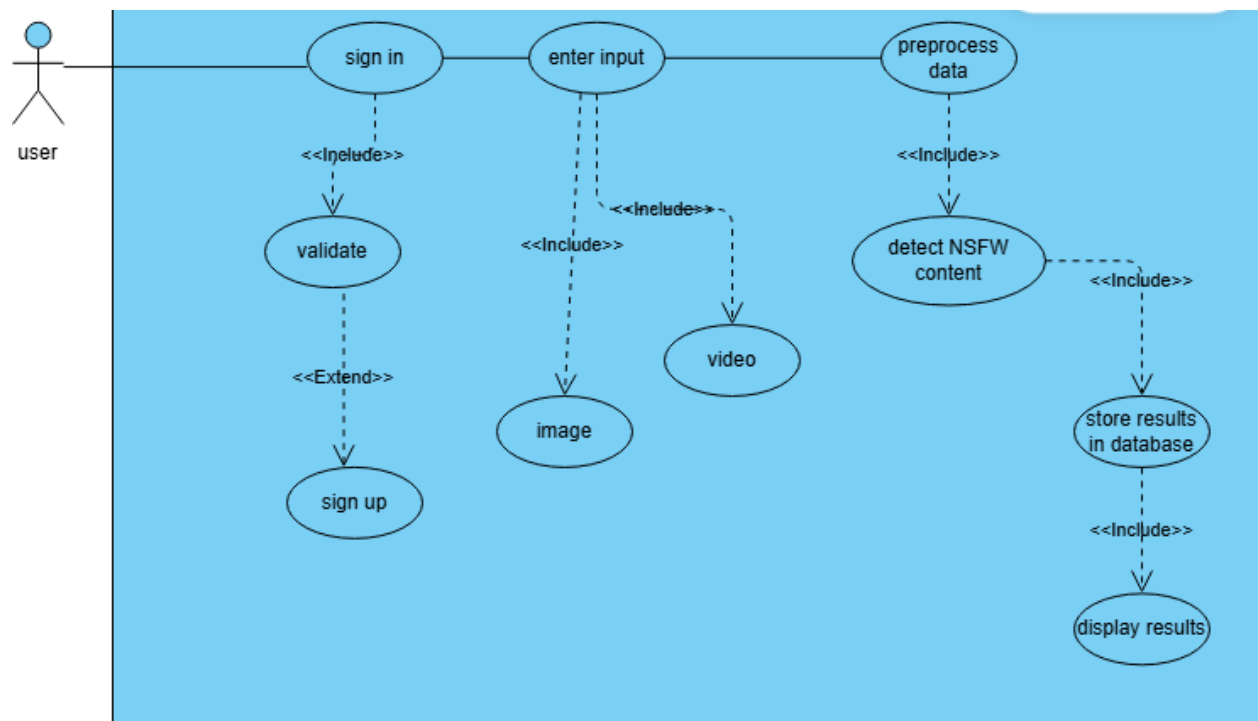


Figure 4.3.1. Use Case Diagram

4.4. Activity Diagrams

4.4.1. Activity Diagrams

4.4.1.1. Input Activity Diagram

After user authentication, if the session is established, the user will be allowed to enter an input in the form of images or videos as shown in [Figure 4.4.1.](#)

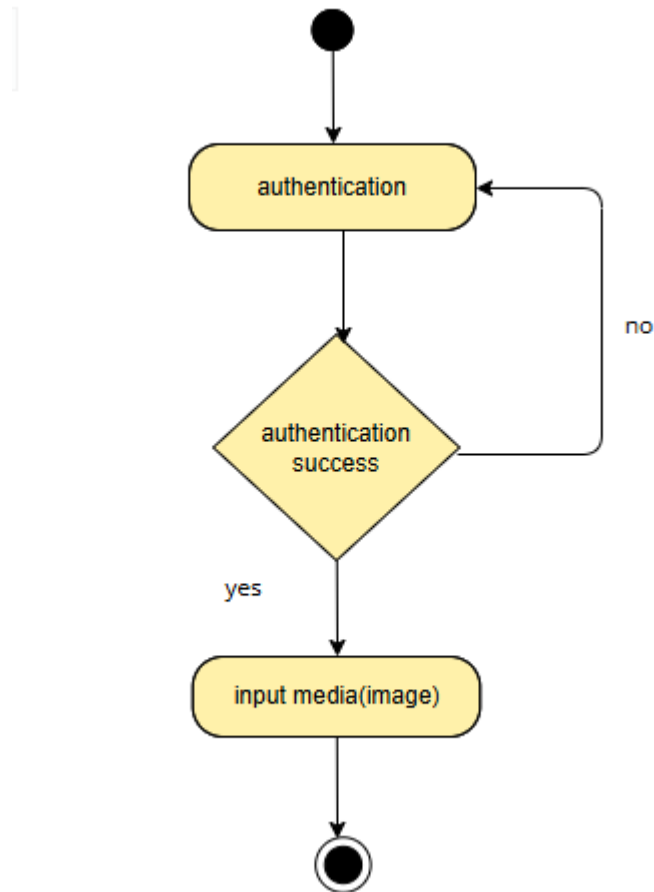


Figure 4.4.1. Input Activity Diagram

4.4.1.2. Media Upload Activity Diagram

Once the user logs into the application, the user can upload either an image or video file. As soon as the file is uploaded, session validation will take place. If the user's session cannot be validated, then the user will be prompted to login back.

Once the validation process is complete, the file will undergo preprocessing, including resizing and normalization. The media file will then be classified using the NSFW detection model as shown in [Figure 4.4.2](#):

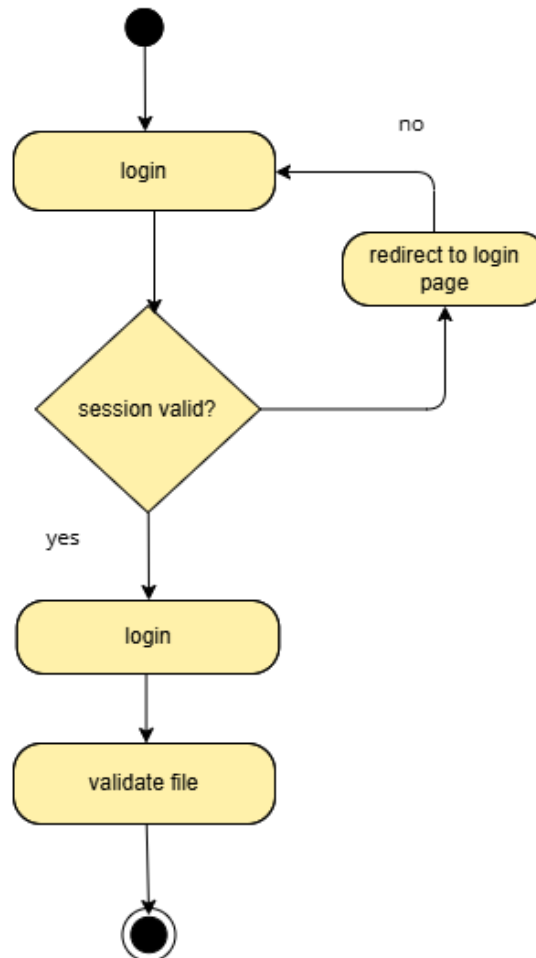


Figure 4.4.2. Media Upload Activity Diagram

4.4.1.3. NSFW Analyzer Activity Diagram

Once the media file is uploaded and preprocessed successfully, the following step is processing of the media file with the help of the deep learning model for identifying any NSFW content in it. The deep learning algorithm then processes the content and categorizes it as being Safe or containing inappropriate content (NSFW). This category is then generated and displayed alongside the file on the dashboard. The process then moves to generating an analysis report that will be

shown to the user on the dashboard once stored. In case there is no session or authentication failure at any stage, the user will be redirected back to the login screen, as shown in [Figure 4.4.3](#):

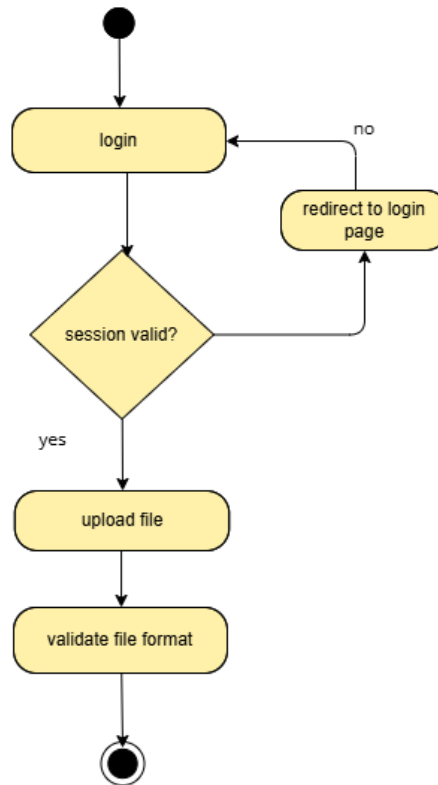


Figure 4.4.3. NSFW Analyzer Activity Diagram

4.4.1.4. Media Analyze Activity Diagram

Once the single image or video file is uploaded, the system stores the file information temporarily and sends it to the deep learning pipeline for analysis.

The NSFW detection model processes the file and produces a classification result is generated and displayed on the dashboard. After that, the analysis report is displayed on the dashboard.

If the user session is invalid or authentication fails, the system redirects the user to the login page, as shown in [Figure 4.4.4](#):

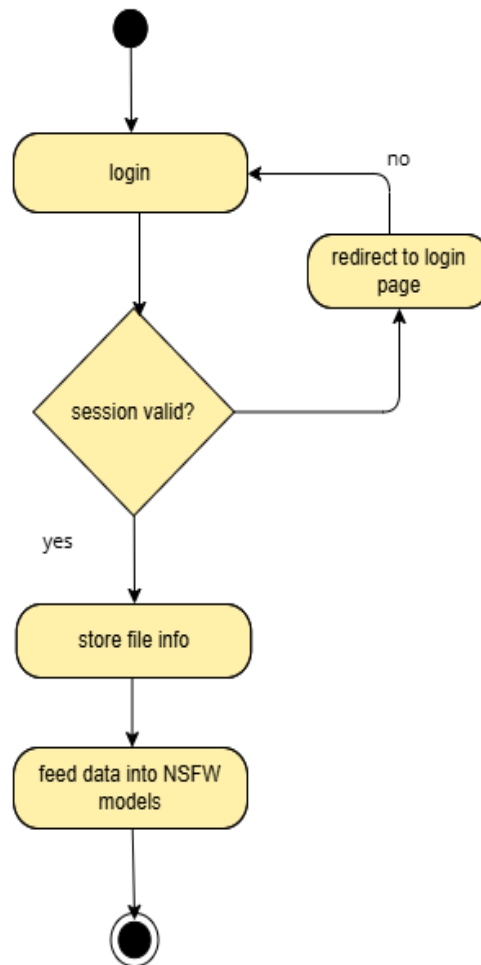


Figure 4.4.4. Media Analyzer Activity Diagram

4.4.1.5. Display Activity Diagram

After the media file has been uploaded, analyzed and the result is generated and displayed on the dashboard, the final classification outcome is displayed on the system dashboard.

The dashboard shows whether the uploaded content is Safe or NSFW, along with any relevant confidence score or analysis details, as shown in [Figure 4.4.5](#):

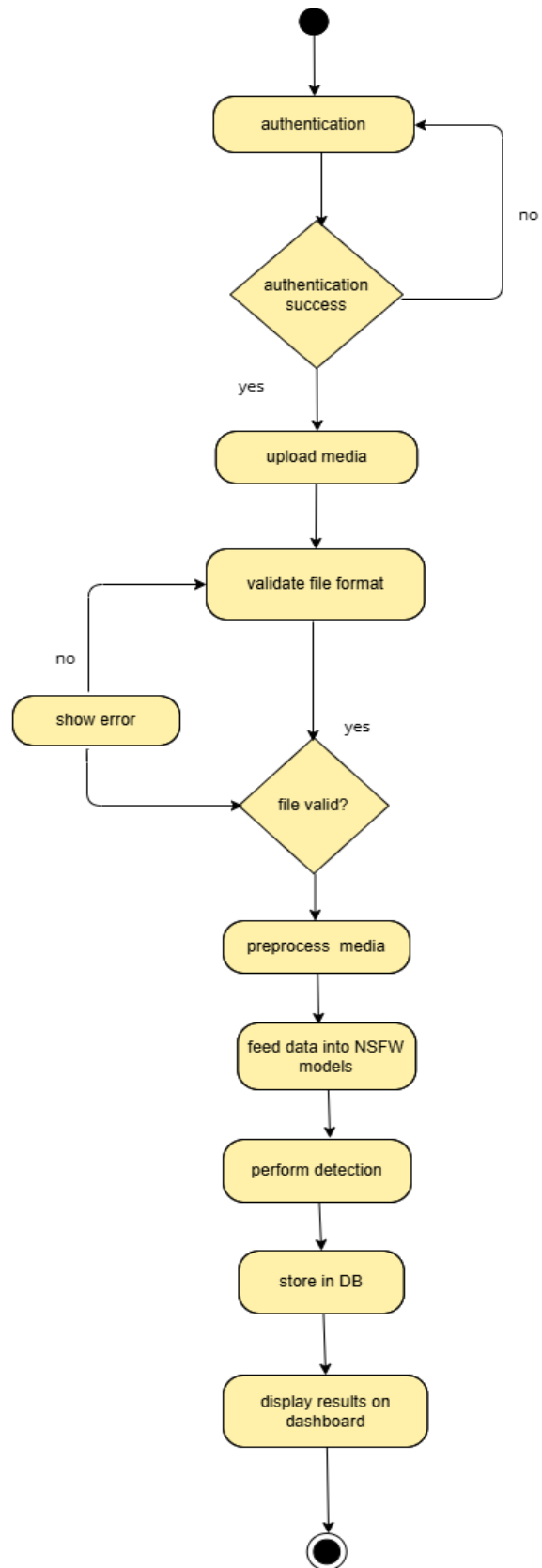


Figure 4.4.5. Display Activity Diagram

4.5. Sequence Diagrams

4.5.1. Login Sequence Diagram

Figure 4.5.6 shows how a user will log in to the system while following the sequence as depicted in the diagram.

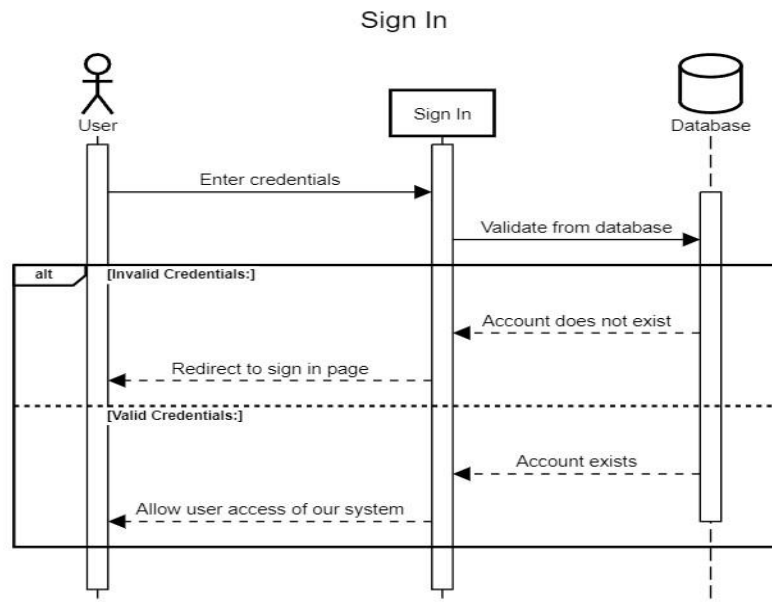


Figure 4.5.6. Login Sequence Diagram

4.5.2. Sign Up Sequence Diagram

Prior to login, the user will sign up first and register him/herself in the database for access to the system functionalities as shown in Figure 4.5.7.

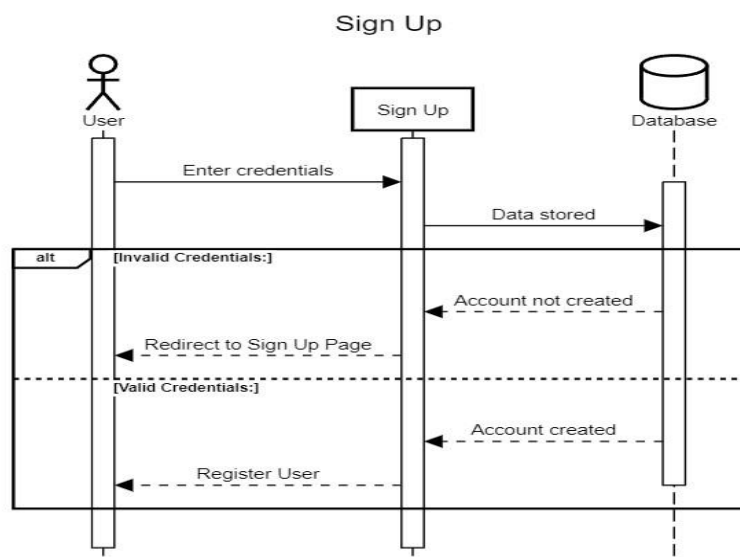


Figure 4.5.7. Sign up sequence diagram

4.5.3. Media Upload Sequence Diagram

Once logged in, the user uploads an image or video file that he or she wants to detect. This file then goes through validation of the current session. If the current session is valid, the file is sent to the preprocessing stage. Preprocessed data is then passed on to the NSFW deep learning model.

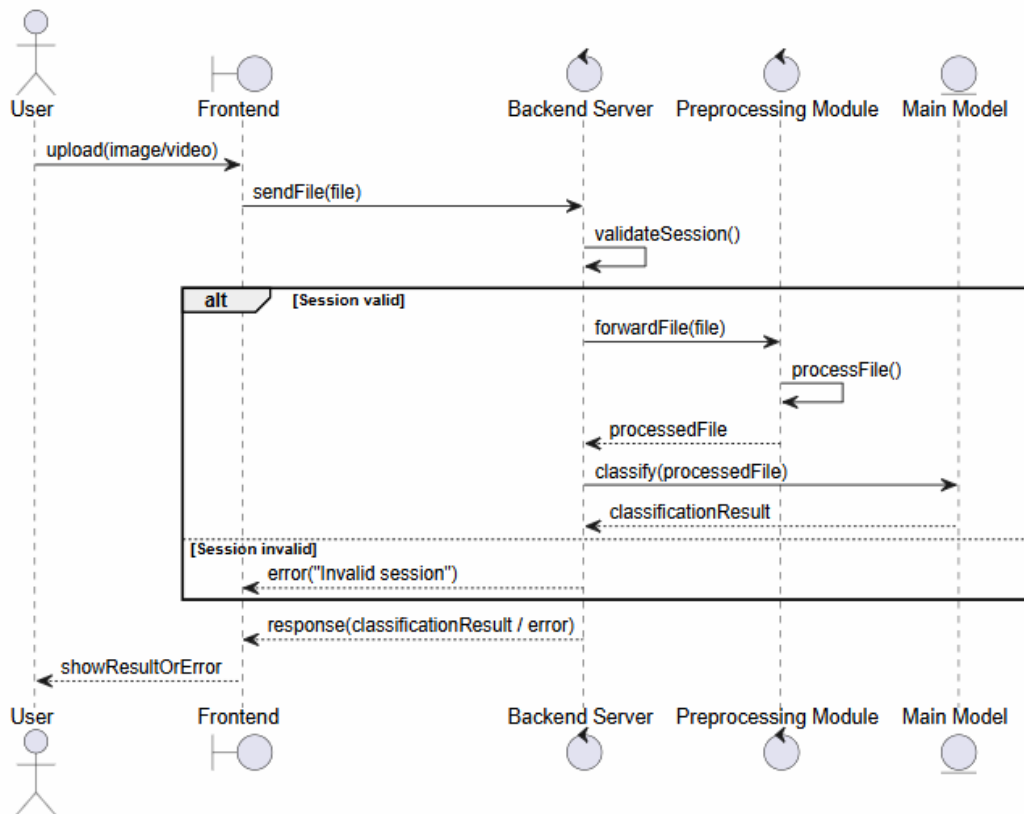


Figure 4.5.8 Media Uploaded Sequence Diagram

4.5.4. Media Processing Sequence Diagram

As represented in [Figure 4.5.8](#), after uploading the media file, the software verifies the validation of the media file format and records its metadata in the database. Next, the processing of the media file takes place, and it is fed to the deep learning model. Finally, the deep learning model analyzes whether it is an NSFW image or not and returns its results as shown in [Figure 4.5.9](#).

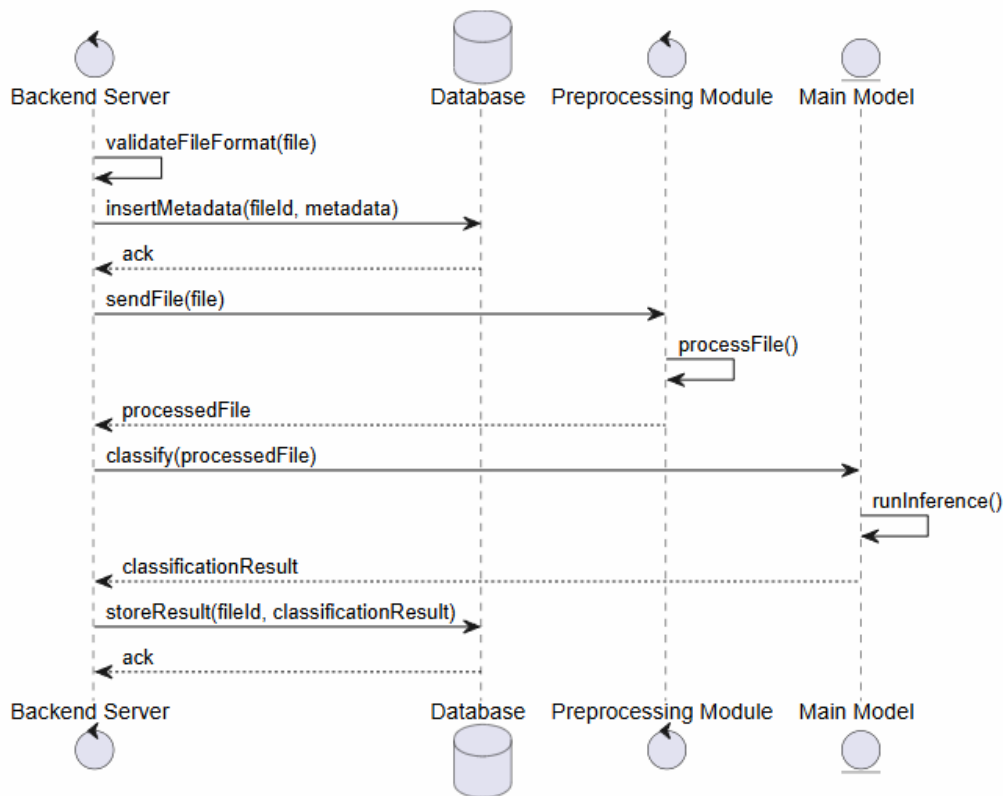


Figure 4.5.9. Media Processing Sequence Diagram

4.5.5. Analyzer Sequence Diagram

As represented in Figure 4.4.10, Once the media file is introduced into the deep learning system, the deep learning system will process the media contents for detecting any NSFW elements in it. Once this is done, the classification results (Safe/NSFW) along with the confidence value obtained from the deep learning system will be returned back to the system and be shown on dashboard.

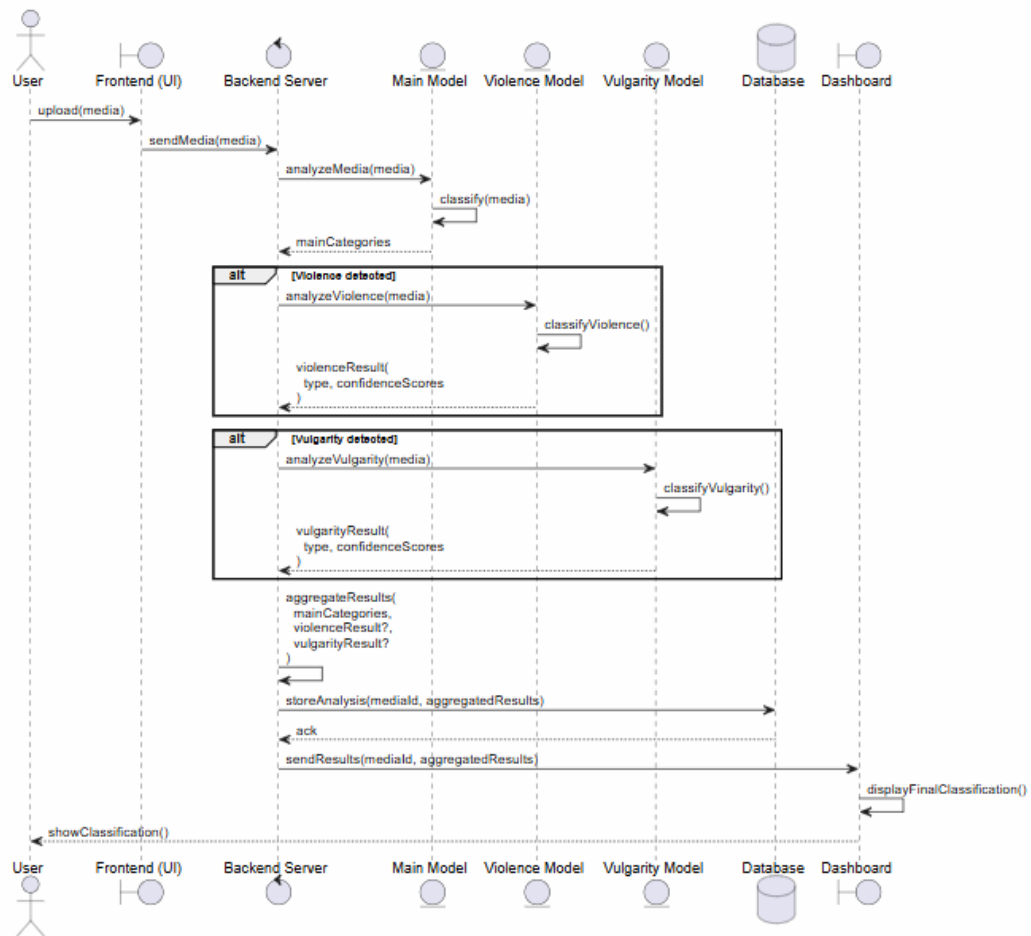


Figure 4.4.10. Analyzer Sequence Diagram

Database Design

4.6. Entity Relationship Diagram

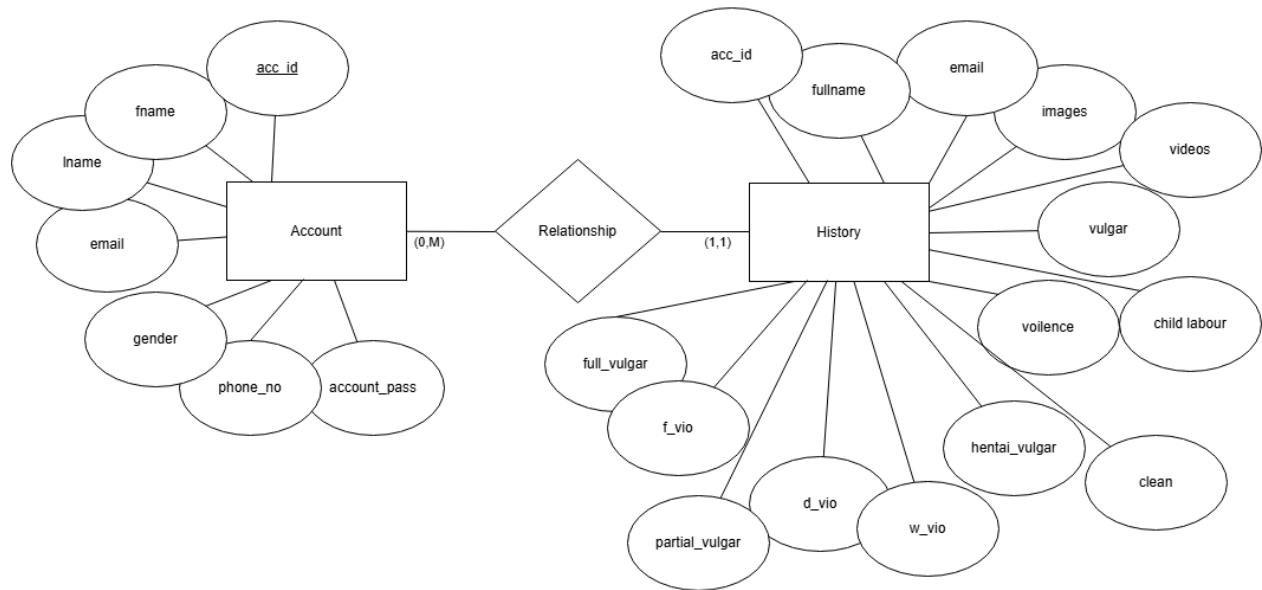


Figure 4.6. ER diagram

The database design presented represents the intended system architecture. In the current implementation phase, lightweight storage mechanisms are used for handling user data and session information.

4.7. GUI Design

The user registration form is shown in [Figure 4.7.1](#). After filling out this form, the user's information is stored by the system, and the user is automatically redirected to the Login page or dashboard flow to access the system.

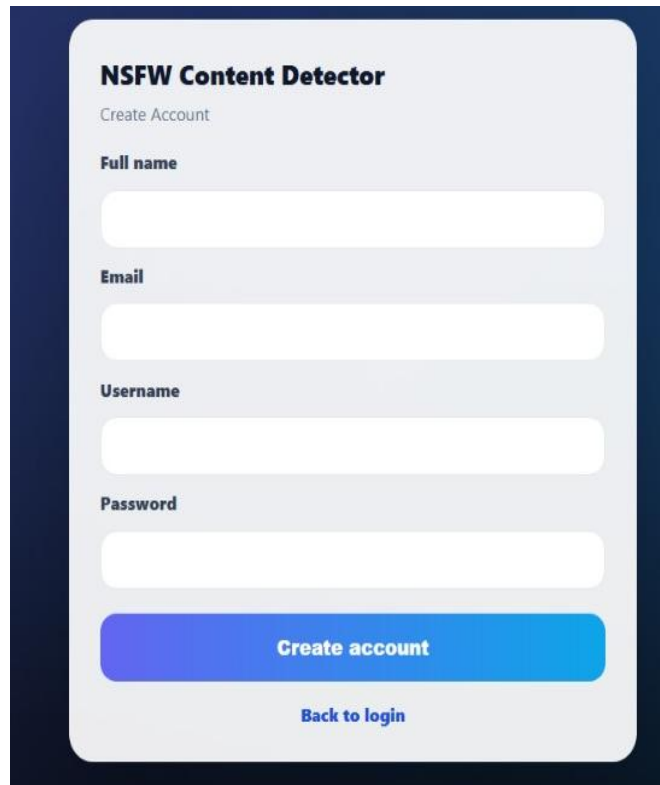
The image shows a sign-up form for 'NSFW Content Detector'. The form is titled 'NSFW Content Detector' and 'Create Account'. It contains four input fields: 'Full name', 'Email', 'Username', and 'Password'. Below the fields is a large blue button labeled 'Create account' and a smaller blue link labeled 'Back to login'.

Figure 4.7.1. Sign Up Page

After registering an account, the user can log in through the web page shown in Figure 4.7.2. Upon successful authentication, the user is redirected to the homepage dashboard as shown in Figure 4.7.3:

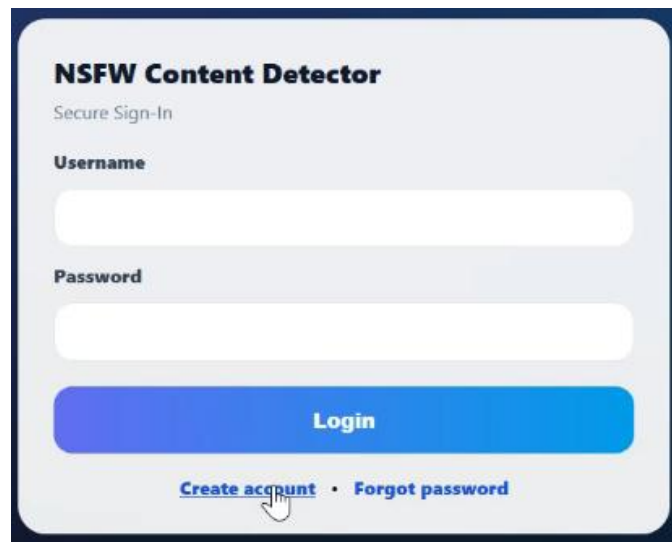
The image shows a login form for 'NSFW Content Detector'. The form is titled 'NSFW Content Detector' and 'Secure Sign-In'. It contains two input fields: 'Username' and 'Password'. Below the fields is a large blue button labeled 'Login'. At the bottom, there are two blue links: 'Create account' and 'Forgot password', separated by a dot. A mouse cursor is pointing at the 'Create account' link.

Figure 4.7.2. Log in Page

The dashboard provides an overview of the system status, supported input formats, and the detection model being used. It displays whether the backend server is online and lists the supported image formats (jpg, png, gif) and video formats (mp4, avi, mkv, mov). The dashboard also indicates that the system uses CLIP Zero-Shot for content analysis.

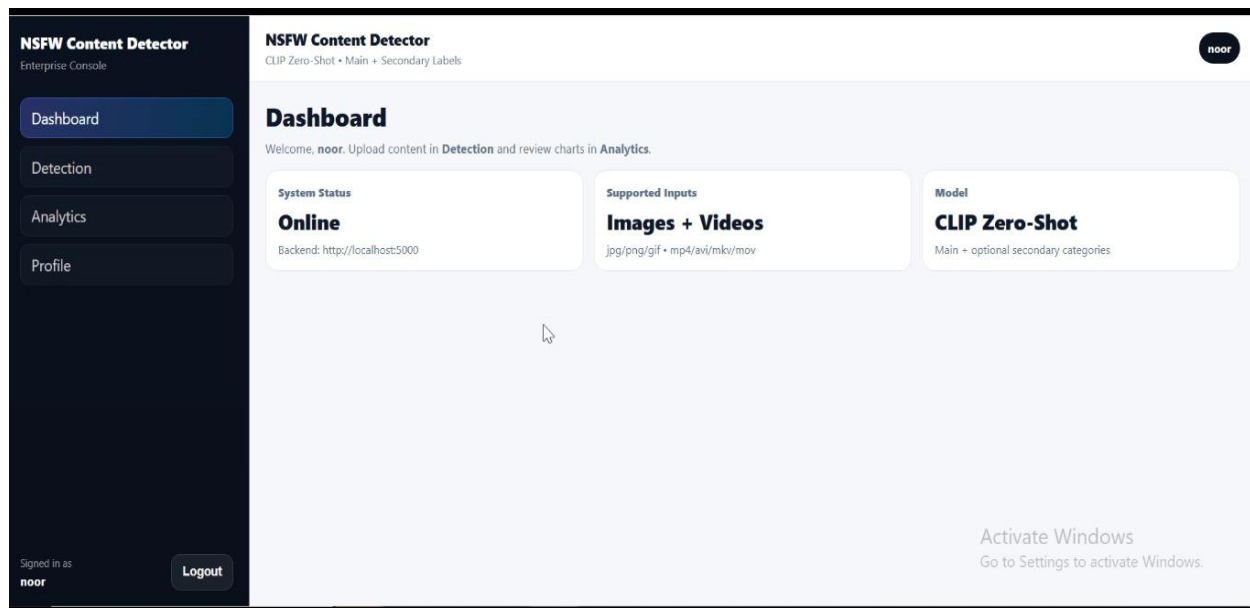


Figure 4.7.3. Dashboard

To perform detection, the user navigates to the Detection module, where they can upload an image or video file directly from their device. Once uploaded, the media file is processed by the backend. In the case of videos, frames are extracted using OpenCV before being passed to the pre-trained deep learning models for classification, shown in Figure 4.7.4;

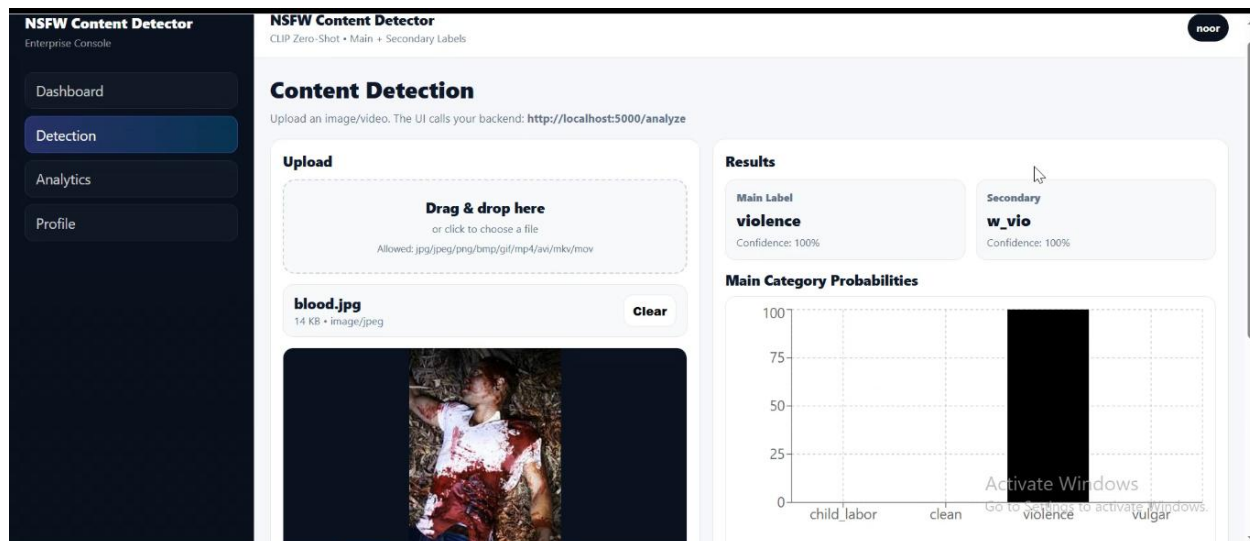


Figure 4.7.4 Dashboard

After analysis, the results are visualized in the Analytics section. By default, previously analyzed data is displayed in graphical form, as shown in Figure 4.7.5 and Figure 4.7.6. The system presents results using both bar charts and pie charts to represent classification distribution in different visual formats.

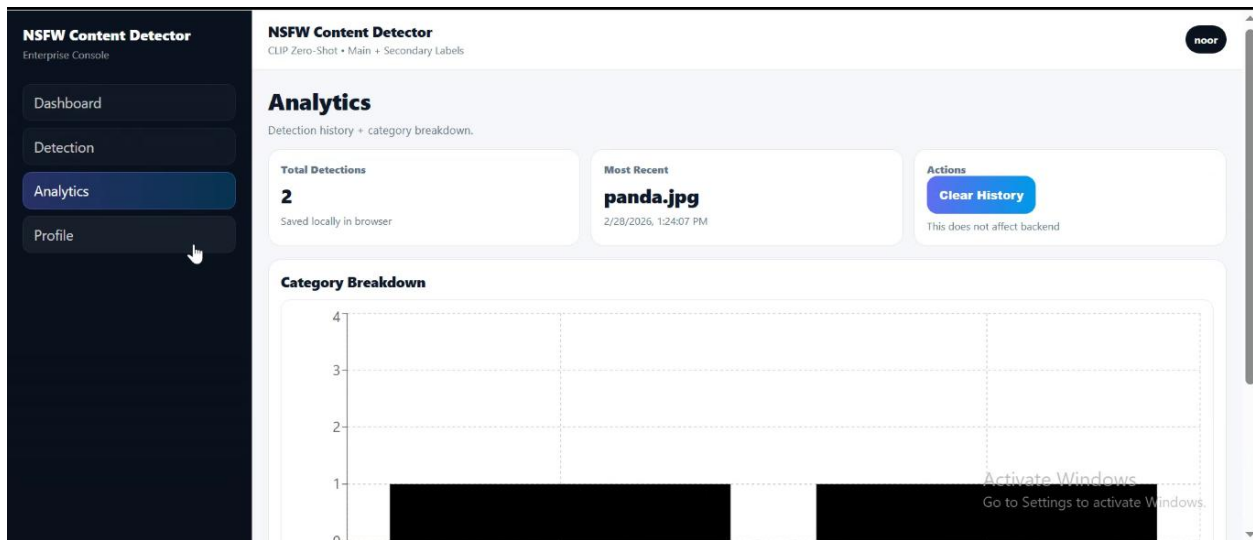


Figure 4.7.5. Analytics section 1

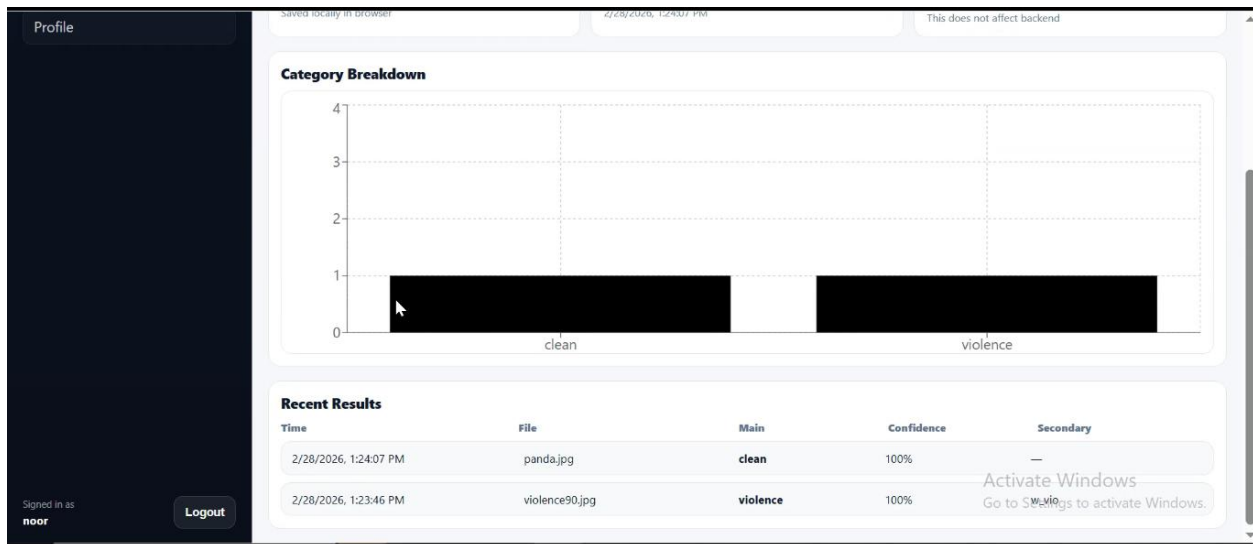


Figure 4.7.6. Analytics section 2

Figure 4.7.7 shows the detailed subdivision of classifications into main categories and secondary subclassifications generated by the models. Figure 4.7.8 presents the complete graphical user interface (GUI), including the navigation sidebar, detection module, analytics dashboard, and profile management section, providing a comprehensive and user-friendly interface for content monitoring.

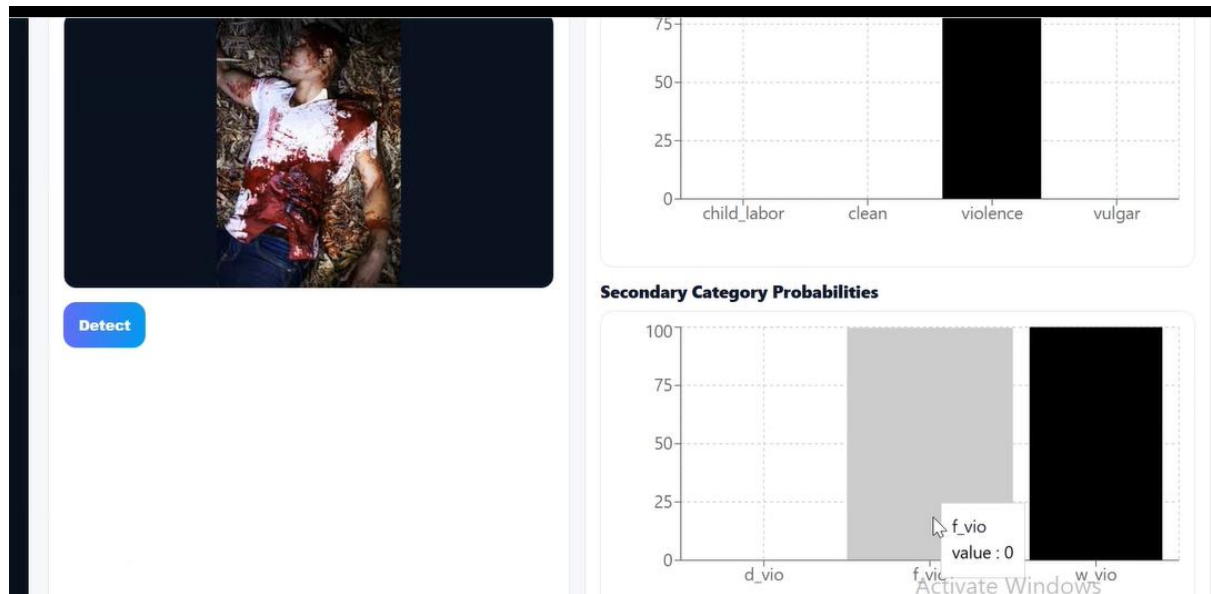


Figure 4.7.7. Subdivision of classifications

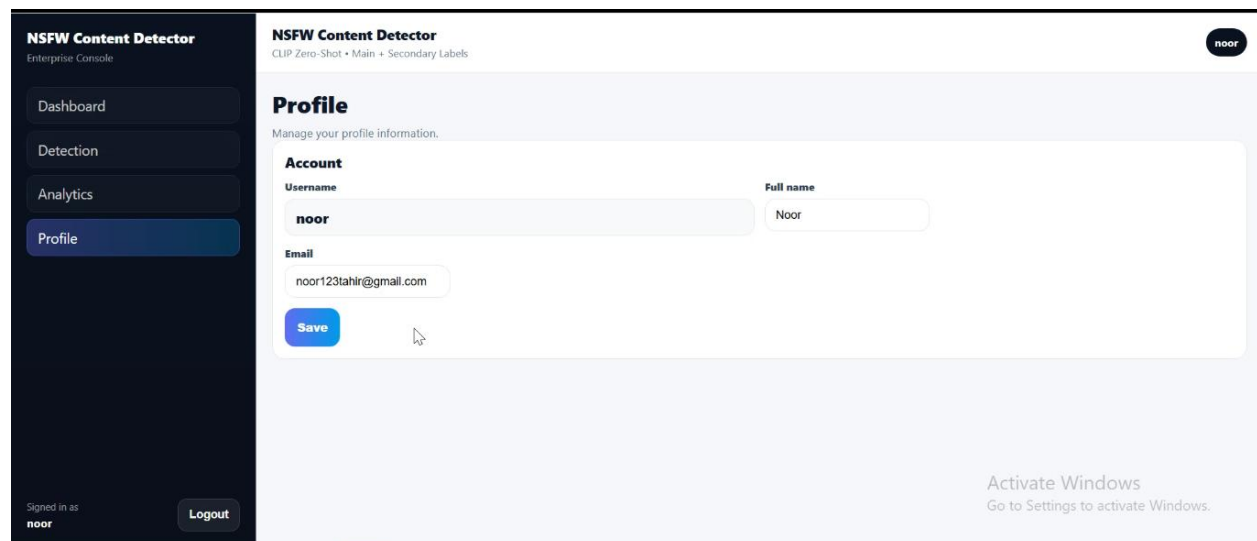


Figure 4.7.8. Graphical user interface (GUI)

Chapter 5

System Implementation

5.1. System Architecture

The NSFW Content Detection system is architected as a web-based application that enables users to upload their images and video clips. This system comprises three major components: the front end of the application, a backend server, and the pretrained deep learning model for analysis of the data uploaded.

The interaction between the user and the system happens through the web application that enables the uploading of files. The backend of this system employs OpenCLIP technology to analyze images and videos based on prompt-based zero shot classification to determine any inappropriate content.

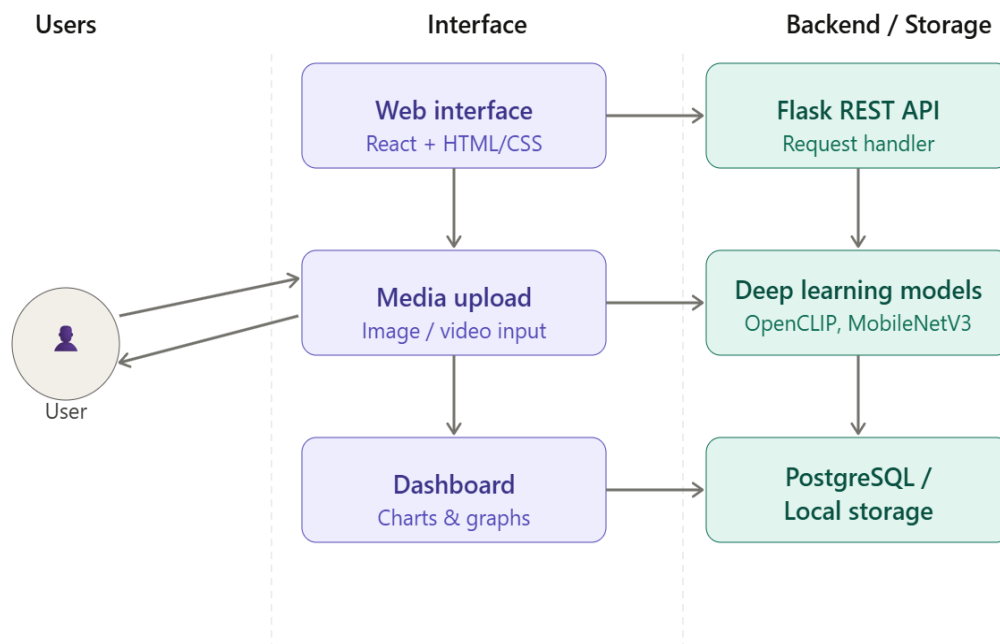


Figure 5.1. High-level System Architecture

5.1.1. System Internal Components

Table 11. System Internal Components

Component Type	Component Name
Internal	Front-end, Image/Video Analysis, User Authentication
Processing	Pre-trained Model (OpenCLIP ViT-B/32), OpenCV
Storage	Browser Local Storage / Application Session Storage

The functionalities of the system components are as follows:

Image/Video Analysis:

The module deals with the image/video analysis aspect. This involves detecting any NSFW content in the uploaded image or video by means of deep learning models that have been pretrained. In case of the video, the extraction of the frames is done via the use of the OpenCV library prior to analyzing the same.

User Authentication:

This module is responsible for handling the registration and logging-in of the users in order to limit access to the system. Upon registration, the user account details are saved within the application. During the authentication process, the system checks the credentials of the person trying to access the system. Upon successful verification, the individual is given access to the application components such as media uploads, dashboard access, and analytics among others. However, upon unsuccessful verification, the user will be notified via an error message.

Front-end:

This provides a platform through which the user interacts with the system by uploading media content, analyzing data, and viewing the analysis result through the dashboard.

Media Upload Module:

The media upload module allows the user to upload his images and videos. This is verified for its correct format and passed on to the backend.

Communication between Components:

Frontend will initiate the request to Backend using HTTP methods (GET/POST). Backend will process the data and send it back to the Frontend.

5.2. Processing Logic / Algorithms

5.2.1. Image/Video Analyzer:

The Image/Video Analyzer is the main part of the system responsible for determining if the uploaded media content is appropriate or inappropriate (contains NSFW elements). This component employs pre-trained deep learning models rather than training the models from scratch; thus, the system works faster and more efficiently.

Primarily, the system utilizes OpenCLIP (ViT-B/32) for zero-shot classification. Models used for the process can understand what is shown in the picture and classify the visual content correctly without additional training.

Firstly, the process begins with the uploading of an image or video via the web interface. In case of an image, the uploaded picture undergoes further processing. In case of a video, the system extracts frames using the OpenCV library. Frames are essentially separate still images captured from the video. Afterwards, all images and extracted frames are resized to match the dimensions accepted by the pre-trained models e.g., 224×224 pixels, and the data normalization process occurs. Once preprocessing is completed, the image or frames are passed into the main classification model. This model classifies the content into four main categories:

- Violence
- Vulgar
- Child Labor
- Clean

This first step acts as a primary filter. If the content is classified as Child Labor or Clean, the system directly finalizes the result without further processing. However, if the content is classified as Violence or Vulgar, the system performs a second level of analysis using secondary prompt-based subclassification.

- For Violence:
 - Physical Violence
 - Weaponized Violence
 - Domestic Violence
- For Vulgar:
 - Full Nudity

- Partial Nudity
- Hentai

This multi-level classification approach helps improve accuracy and provides more detailed insights about the content.

The models perform inference using Torch, which allows fast and efficient prediction. Once the analysis process is finished, the final output is produced and Last but not least, the output will be saved into the PostgreSQL database or on local storage for now and forwarded to the front-end side. Finally, the user will have access to the output from the dashboard.

5.2.2. Main Model

The Main Model is the core model used for classifying any image uploaded or extracted video frame. The model classifies the content of the image/video into one of the following four categories:

- Violence
- Vulgar
- Child labor
- Clean

The OpenCLIP (ViT-B/32) framework is used in the zero-shot approach to classify images. Text prompts (labels) are used to compare against image features instead of the traditional training process.

After a user has uploaded a video/image, the following process takes place:

1. For videos as an input source, frames are extracted using OpenCV.
2. Input Frame/Image will be downsized and standardized.
3. The input frame will then be passed into the model.
4. The most related category is chosen based on similarities in image feature-text prompt.

If the classification falls under the Child Labor or Clean category, the final result is saved. Otherwise, if the prediction falls under the Violent/Vulgar category, the next models are consulted for more accurate results.

5.2.3. Violence Model

The Violence Model is a secondary classification stage that is used only when the Main Model predicts the category Violence.

This model performs deeper classification into the following subcategories:

- Physical Violence
- Weaponized Violence
- Domestic Violence

The system analyzes the input image or extracted video frames using prompt-based zero-shot classification. It identifies patterns such as aggressive actions, presence of weapons, or harmful interactions between individuals.

This step improves the accuracy and provides more detailed classification compared to the main model.

5.2.4. Vulgarities Model

The Violence Model is a secondary stage of classification that is only applied when the Main Model identifies the category Violence. The Violence Model applies deeper classification to the following subcategories:

- Full Nudity
- Partial Nudity
- Hentai

This stage of the system applies prompt-based zero-shot classification to the input image or the selected video frames. The system can detect patterns including violent behavior, weapons present, or interactions that harm others. This step helps improve the accuracy of the classification process.

5.2.5. Processing Pipeline

The complete process flow of the system is described as follows:

1. The user uploads an image or a video.
2. The video is split into frames using OpenCV (wherever necessary).
3. Pre-processing of images/frames takes place (resizing and normalization).
4. Classification by Main Model.
5. Classification by secondary models (whenever necessary).
6. Final result is generated.
7. The output is generated and displayed via application UI.
8. Dashboard display of the output.

5.2.6. Model Inference

The Torch-based inference is used to execute the pre-trained models. The models are executed quickly and efficiently without any need for training since the models have been trained using massive data sets.

5.3. User Authentication

User authentication can be accomplished using the Flask framework, which will be responsible for securing the access rights of the users to the system. The module provides support for both registration and login processes. This is achieved by verifying the validity of credentials provided by the user such as email and password with regards to entries in the PostgreSQL database or on local storage.

In the case of registration, the credentials will be stored in the database. On login, the entered email and password will be checked in order to authenticate the user's identity. Once this step is complete, a secure session will be created that will help in the management of future requests. Access to some functionalities, such as media uploads and dashboard access, will only be provided after verification of the session and failure to provide authentication details will deny users access to the system.

5.4. Tools/Technologies Used

The following are the tools and technologies utilized in the development of the web application.

5.4.1. Python

Python is preferred for the use in back-end languages because of its simplicity and ease of use and availability of multiple libraries. It can be classified as a high-level programming language and enables rapid development. Libraries of Python have been found helpful in applications like machine learning, image processing, and even web development.

5.4.2. Machine/Deep Learning Frameworks

The task of executing the pre-trained model in order to carry out inference is done by using PyTorch. For inappropriate content detection through zero-shot classification, the system makes use of OpenCLIP (ViT-B/32).

5.4.3. Storage

The system can store information about user sessions and analyses using the frontend's application-based storage options. It ensures that the system is able to keep track of user-state information and earlier output results for display on the dashboard.

5.4.4. Flask

The Flask web framework for Python is used in creating the backend of the application. It takes care of making the API calls, handling user login and registration, uploading media content, interacting with the database, and machine learning modules.

5.4.5. PyCharm

PyCharm is a Python Integrated Development Environment (IDE) used for developing and managing the backend code. It provides tools for debugging, code completion, and project management, making development more efficient.

5.4.6. React

An open-source JS library used to develop the front end of the web application is known as React. Through the React framework, we can develop interactive UIs. The components of the front end that include the dashboard, the upload interface, and the analytics are developed using it.

5.4.7. JavaScript

JavaScript is employed in the front end to write the logic and interactive elements of the application.

5.4.8. HTML

HTML (HyperText Markup Language) is used to structure the content of the web pages. It defines the layout and elements such as forms, buttons, and text displayed in the application.

5.4.9. CSS

CSS (Cascading Style Sheets) is used to design and style the web pages. It controls the layout, colours, fonts, and responsiveness of the user interface.

5.4.10. OpenCV

The OpenCV library is a computer vision toolkit that helps process video feeds. The software extracts frames from videos that have been uploaded by the user and analyzes each frame separately using the deep learning algorithms.

5.4.11. Model Inference (Torch)

Inference using the Torch library is adopted in the implementation of the pre-trained models. This allows rapid generation of predictions without the need to train the model again.

Chapter 6

System Testing and Evaluation

In this chapter, we will review the performance of our web application as well as evaluate its effectiveness. The evaluation of web applications is accomplished by making use of a wide variety of tools and approaches. The primary objective in developing this system was to analyze videos and images and to determine whether it contains negative content such as vulgarity, violence, and child labor. The testing of the entire system and all of its components is covered in this chapter. If the system produces findings that are in accordance with the requirements that were set, then the testing may be considered to have been successful.

6.1. Graphical User Interface (GUI) Testing

Testing the Graphical User Interface (GUI) is a procedure that evaluates how easily a user can interact with a web application by utilizing the interface provided by the online application. The graphical user interface must be user-friendly in order for a user who is unfamiliar with web applications to readily comprehend how to use the web application by only reading the text or looking at the graphics on the buttons. The GUI is dependent on human interaction with the application by clicking different options and buttons. GUI also includes color schemes, design patterns, fonts, and ease of use which is only responsible for better human and computer interaction.

6.2. Usability Testing

During the usability testing, the goal is to verify that a user with no prior technical skills can use the web application without any difficulty. Testing an application's usability guarantees that it is straightforward to operate.

6.2.1. Register Test Case

The register test case aims to ensure that the process is smooth and hassle-free. By testing the registration page thoroughly, we can identify any potential issues that might hinder users from creating an account, such as incorrect form fields or missing information. It is a simple form-type registration page, as shown in [Table 12](#):

Table 12. Register Test Case

Test Case ID: UTC - 001
Test Objective: Register with correct credentials.
Test Case Description: To create an account on the website.
Preconditions:
1. Internet connection 2. Web browser 3. User must not already have an account with the same email 4. Registration form with required fields is available
Environment:
1. Testing Devices: Desktop Computer, Laptop 2. Environment Variables: Internet, Web Browser
Test Data: Email, password (and confirm password), first name, last name, gender, phone number
Initial State: Use valid credentials
Test Actions / Steps:
1. Open web browser 2. Click on “Register” 3. Enter valid user details (name, email, password, etc.) 4. Click on Register
Expected Output: User account should be successfully created and redirected to the Login page
Actual Output: User account was successfully created and redirected to the Login page
Status: Pass

6.2.2. Login Test Case

The login test ensures that there is restricted access to the system only by registered users. The user logs in using his/her registered email address and password. In case the email address and password provided by the user are right, then the user gains access and is redirected to the dashboard. However, if wrong details are provided, an error message pops up. As shown in [Table 13](#):

Table 13. Login Test Case

Test Case ID: UTC - 002
Test Objective: Login with authorized credentials.
Test Case Description: To login to the website and access system features.
Preconditions:
1. Internet connection 2. Web browser 3. User must already have a registered account 4. Login form should be available
Environment:
1. Testing Devices: Desktop Computer, Laptop 2. Environment Variables: Internet, Web Browser
Test Data: Email and Password
Initial State: Use valid registered credentials
Test Actions / Steps:
1. Open web browser 2. Click on “Login” 3. Enter registered email 4. Enter correct password 5. Click on Sign In

Expected Output: User should be successfully logged in and redirected to the dashboard
Actual Output: User successfully logged in and redirected to the dashboard
Status: Pass

6.2.3. Input Test Case

The test evaluates the ability of the software to receive proper user input for analysis purposes. Our modified software now enables the user to upload images or videos for analysis. The software analyses the input and passes it on to the back end for further processing. As shown in [Table 14](#):

Table 14. Input Test Case

Test Case ID: UTC - 003
Test Objective: Provide input for analysis.
Test Case Description: To give input (image, video) to the system for analysis.
Preconditions:
1. Internet connection 2. Web browser 3. User must be logged into the system 4. Input fields or upload options should be available
Environment:
1. Testing Devices: Desktop Computer, Laptop 2. Environment Variables: Internet, Web Browser
Test Data: Image file / Video file
Initial State: User is on dashboard
Test Actions / Steps:
1. Upload an image or video. 2. Click on “Analyze” button

Expected Output: Input should be successfully sent to the backend for processing
Actual Output: Input was successfully processed by the system
Status: Pass

6.2.4. Processing Test Case

The process test case verifies whether the system processes the user inputs correctly when they are entered. In the upgraded system, the user input is sent to the backend for analysis using machine learning algorithms. The process test case will verify whether the system can trigger the process and obtain results, as shown in [Table 15](#):

Table 15. Processing Test Case

Test Case ID: UTC - 004
Test Objective: Process user input for analysis.
Test Case Description: To ensure that the system processes uploaded media and sends it to the analysis module.
Preconditions:
1. Internet connection 2. Web browser 3. User must be logged in 4. Input must be provided
Environment:
1. Testing Devices: Desktop Computer, Laptop 2. Environment Variables: Internet, Web Browser
Test Data: Image / Video
Initial State: Input is ready for processing
Test Actions / Steps:
1. Provide input 2. Click analyze/search

Expected Output: System should start processing and send data to ML models
Actual Output: System successfully started processing
Status: Pass

6.2.5. Analyzer Test Case

The purpose of this test is to check that the models function accurately. If the user uploads an image, it gets analyzed directly. However, if the user uploads a video, it first gets segmented using OpenCV and then each frame is analyzed. In the end, a classification is obtained that can be either Violence, Vulgar, Child Labor, or Clean. As shown in [Table 16](#):

Table 16. Analyzer Test Case

Test Case ID: UTC - 005
Test Objective: Analyze image/video content.
Test Case Description: To check whether deep learning models correctly analyze the media content.
Preconditions:
1. Internet connection 2. Web browser 3. User must be logged in 4. Input data must be available
Environment:
1. Testing Devices: Desktop Computer, Laptop 2. Environment Variables: Internet, Web Browser
Test Data: Image or video data
Initial State: Model ready for execution
Test Actions / Steps:

1. Upload image/video 2. System processes data 3. Model generates classification
Expected Output: System should classify content (Violence, Vulgar, Child Labor, Clean)
Actual Output: System successfully classified the content
Status: Pass

6.3. Software Performance Testing

The performance test is used to ensure that the efficiency and speed of the system are satisfactory. The performance test verifies if the system is capable of processing data within an acceptable period of time. As far as our system is concerned, the performance test focused on video and image processing speed.

6.4. Performance Testing

This evaluation considers the efficiency of the system. The time taken by the system to analyze videos was determined. The system was capable of analyzing about one second of video footage within five seconds. Nevertheless, the efficiency of the system might be lower when dealing with longer videos since frame extraction and frame-wise classification consume more time, as shown in [Table 17](#):

Table 17. Performance Testing

Test Case ID: PTC - 001
Test Objective: Test system processing speed.
Test Case Description: To check how fast the system analyzes video content.
Preconditions:
1. Internet connection 2. Web browser 3. User must be logged in

4. Input video must be available
Environment:
1. Testing Devices: Desktop Computer, Laptop
2. Environment Variables: Internet, Web Browser
Test Data: Video (30 fps)
Initial State: Model ready for execution
Test Actions / Steps:
1. Upload video 2. Click analyze 3. Measure processing time
Expected Output: System should analyze 1 second of video within ≤ 7 seconds
Actual Output: System analyzed 1 second of video in approximately 5 seconds
Status: Pass

6.5. Evaluation Metrics for Deep Learning Models

6.5.1. Main Model

Main Model performs the classification of all pictures and video frames into four classes, namely Violence, Vulgar, Child Labor, and Clean. All inputs have to go through this model before any other operations can be performed on them.

It was possible to process about 1 second of video within 5 seconds of testing. However, the processing efficiency is likely to reduce in longer videos because of the increased time required for frame extraction and frame-wise classification.

Training Results (Main Model):

Table 18. Training Results (Main Model)

Metric	Value
Accuracy	86.3%
Loss	0.366
Precision	88.4%
Recall	84.1%
F1 Score	86.2%

Validation Results (Main Model):

Table 19. Validation Results (Main Model)

Metric	Value
Validation Accuracy	65.9%
Validation Loss	0.8651
Validation Precision	68.4%
Validation Recall	64.2%
Validation F1 Score	66.2%

Analysis:

The gap between training and validation accuracy shows overfitting, meaning the model learns training data well but struggles with new data.

6.5.2. Violence model:

The Violence Model is used when the Main Model detects violence. It further classifies the content into Physical Violence, Weaponized Violence, and Domestic Violence.

This model also used MobileNetV3 and faced similar issues due to limited datasets, especially for domestic violence.

Training Results (Violence Model):

Table 20. Training Results (Violence Model)

Metric	Value
Accuracy	82.6%
Loss	0.436
Precision	82.8%
Recall	83.2%
F1 Score	84.9%

Validation Results (Violence Model):

Table 21. Validation Results (Violence Model)

Metric	Value
Validation Accuracy	65.75%
Validation Loss	0.7127
Validation Precision	66.7%
Validation Recall	64.6%
Validation F1 Score	65.6%

Analysis:

Performance is moderate and affected by data scarcity and imbalance.

6.5.3. Vulgarly Model

The Vulgarly Model classifies content into Full Nudity, Partial Nudity, and Hentai. It performed better than other models because of the availability of a structured dataset.

Training Results (Vulgarly Model):

Table 22. Training Results (Vulgarly Model)

Metric	Value
Accuracy	86.23%

Loss	0.3486
Precision	87.15%
Recall	85.26%
F1 Score	86.2%

Validation Results (Vulgarity Model):

Table 23. Validation Results (Vulgarity Model)

Metric	Value
Validation Accuracy	87.53%
Validation Loss	0.3147
Validation Precision	88.18%
Validation Recall	86.48%
Validation F1 Score	87.32%

Analysis:

This model performed well due to better dataset quality and clear visual patterns.

6.6. Use of Additional Models (Improvement):

To overcome these limitations, we integrated additional pre-trained models into our system:

Models Added:

- **OpenCLIP (ViT-B/32):** To achieve zero-shot classification and improved semantic understanding.
- For the deployed system, the classification is done using the zero-shot classification technique with the help of OpenCLIP (ViT-B/32). Due to the nature of the classification pipeline where there is no task-specific training required for the classification using any labeled datasets, tables like training, and validation accuracy, precision, loss, etc., for the trained MobileNetV3 can't be applied to the deployed system.

- Instead, performance of the system was tested by practical application to the image/video files that had been uploaded into the software. The system normally performed fairly well with regard to the four primary categories: Violence, Vulgar, Child Labor, and Clean. Some shortcomings have been noted in the process of testing, however. For example, in certain instances, the system would classify a child learning or playing as "Child Labor" instead of categorizing the image as "Clean". Furthermore, the system may lack accuracy when processing long videos since it analyzes individual frames in the video only.
- Further improvement of the system behavior could be achieved by improving the design of prompts and subclassification logic. It should be noted that since the deployed system uses zero-shot classification, its accuracy greatly relies on the quality of prompts and contextual images.

6.7. Compatibility Testing

The compatibility test guarantees the proper working of the software across different platforms. Our system was put under tests on different computers as well as browsers. The outcome revealed that the system runs effectively on all platforms without problems.

6.8. Exception Handling

The web application consists of multiple modules including user authentication and deep learning-based content analysis. Each module has its own exception-handling mechanism to ensure smooth execution. When incorrect credentials or invalid inputs are provided, the system generates proper error messages instead of crashing. The system also handles errors related to invalid image/video inputs, unsupported formats, and model prediction failures.

Since our project uses a pre-trained deep learning classification pipeline, exception handling is also applied during model execution. If the system fails to process an input or produces an error during analysis, an appropriate error message is displayed.

6.8.1. Register Test Case:

This test case validates the functionality of the system on registration by an invalid or partially entered details by users. This test case will check whether proper validation errors will be shown to the user in case of invalid email formats, poor password strength, empty fields, or invalid phone numbers, as shown in [Table 24](#):

Table 24. Register Test Case

Test Case ID	ETC - 001
Test Objective	Register with incorrect/invalid credentials
Test Case Description	User attempts to create an account using invalid inputs

Preconditions	1. Internet connection 2. Web browser 3. Registration form available
Environment	Desktop/Laptop, Internet
Test Data	Invalid email, password, name, gender, phone
Initial State	Invalid credentials
Test Steps	1. Open Register page 2. Enter invalid data 3. Click Register
Expected Output	System should display validation errors
Actual Output	Proper error messages displayed
Status	Pass

6.8.2. Login Test Case:

This test case tests how the system behaves in case of wrong username and password entries. The test case ensures that there is no way for the system to be accessed by an unauthorized user, as shown in [Table 25](#):

Table 25. Login Test Case

Test Case ID	ETC - 002
Test Objective	Login with invalid credentials
Test Case Description	User tries to login with incorrect email/password
Preconditions	User account exists
Environment	Desktop/Laptop
Test Data	Invalid email/password
Initial State	Invalid credentials
Test Steps	1. Open Login page 2. Enter wrong credentials 3. Click Login
Expected Output	Error message displayed

Actual Output	Error message displayed
Status	Pass

6.8.3. Input Test Case:

This test case assesses the ability of the system to cope with any kind of invalid input while uploading content. The test assesses if the system is able to reject invalid formats for any file, or corrupted files, as shown in [Table 26](#):

Table 26. Input Test Case

Test Case ID	ETC - 003
Test Objective	Provide invalid input for analysis
Test Case Description	User uploads invalid or unsupported content
Preconditions	User logged in; upload option available
Environment	Desktop/Laptop
Test Data	Unsupported file format, empty input, corrupted file
Initial State	Invalid input
Test Steps	1. Upload invalid file 2. Click Analyze
Expected Output	System should display file/input error
Actual Output	Proper error message displayed
Status	Pass

6.8.4. Model Processing Test Case:

It makes sure that a valid input (video/image) will be processed by the system appropriately. This test case validates that the input is indeed passed to the Main Model without errors for classification, as shown in [Table 27](#):

Table 27. Model Processing Test Case

Test Case ID	ETC - 004
Test Objective	Verify model processing of valid input
Test Case Description	Check whether the system correctly processes input using deep learning models
Preconditions	Valid image/video uploaded
Environment	Desktop/Laptop
Test Data	Valid image/video content
Initial State	Model ready
Test Steps	1. Upload valid input 2. Click Analyze
Expected Output	System should process input and send it to Main Model
Actual Output	Input processed successfully
Status	Pass

6.8.5. Analyzer Test Case:

It makes sure that the models analyze and classify the input correctly. It validates that the output generated is relevant and accurate according to the input provided. As shown in [Table 28](#):

Table 28. Analyzer Test Case

Test Case ID	ETC - 005
Test Objective	Verify classification accuracy of models
Test Case Description	Test whether models classify content correctly
Preconditions	Valid input available
Environment	Desktop/Laptop
Test Data	Images/videos (Violence, Vulgar, Child Labor, Clean)

Initial State	Models loaded
Test Steps	1. Input data to Main Model 2. Trigger specialized models if needed
Expected Output	Correct classification into categories
Actual Output	Classification generated successfully
Status	Pass

6.9. Load Testing

While performing load testing, the response time of a web application is the primary emphasis. Load testing is used to assess how long it takes for our web application to execute a query in order to get an accurate time estimate.

6.10. Security Testing

The purpose of security testing is to ensure that our web application is not accessible to anybody who is not registered to use it. Access may be achieved by using a login module.

6.11. Installation Testing

The act of putting a web application through its tests on a variety of operating systems is known as installation testing. In order to verify that the web application is compatible with a variety of operating systems, we installed and ran it on multiple systems.

Chapter 7

7.1. Conclusion

Not Safe For Work Content Detector is a software used for recognizing offensive pictures and videos automatically. This technology aims at recognizing sensitive issues such as violence, vulgarity and child labour in an effort to curb spread of offensive materials online. This technology employs machine learning in analyzing pictures and video shots uploaded by users. Contrary to normal computer programs, Social Media Violence, Vulgarity, and Child Labour Analyzer does not require any data crawling as it analyzes uploaded user content via deep learning classifiers, which makes it fast and effective.

Python programming language powers this technology through a machine learning algorithm. The application classifies uploaded content based on its Main Model as Violence, Vulgar, Child Labour and Clean. Depending on the output obtained from the model, the subclasses are then analyzed through the specific models as follows; violence subclassified into Physical Violence, Weaponized Violence and Domestic Violence. Also, the Vulgar information is subdivided as Full Nudity, Partial Nudity and Hentai.

It was noticed during development that zero-shot classification can result in errors in context-sensitive cases. As an illustration, in some cases, children engaged in studies or at play can be mislabeled as child labor as opposed to clean content. In addition, videos can also be lengthy enough for unreliable analysis since at present the approach used relies on frame extractions and frame-wise analysis. This shows the need for better prompt design and data collection in addition to better temporal understanding of videos. The frontend is made easy for users where content can be uploaded easily and results can be obtained effortlessly. Results are then provided clearly to facilitate users' understanding.

In general, the AI-based detection application shows how artificial intelligence can be utilized efficiently in detecting harmful contents. Overall, the use of such an application would prove quite useful in moderating content.

7.2. Disciplined Project Management

The key to the successful completion of a project and most importantly making the correct product is the conduction of proper and disciplined project management, and through the course of the project, we learned the importance of managing the time and the resources simultaneously to meet the schedule and the plan of the respective project and what effect these factors can have if they are not dealt with the cautious effort.

7.3. Team Communication

Another aspect of the importance we got familiar with is the role the communication between the team members which is the primary goal of all projects. The communication should be open, honest, and fair so that everyone in the team can contribute his/her best and never feel like their ideas and contributions are minimized or marginalized. To get the best result, it is important to maintain a good relationship with all the team members.

7.4. Future improvements

However, despite the efficient working of this system, there are some aspects through which it could have been improved further for better results and performance. Some of these aspects include the application of more sophisticated deep learning models as well as bigger and balanced datasets to prevent overfitting. Other ways of improving performance include fine-tuning the already used model, using data augmentation techniques, etc. The shortcoming associated with this project related to accuracy is that the children who are studying or playing are classified under “Child Labor” instead of being classified under “Clean.” Another issue faced by the system is that it does not work properly or becomes inefficient when longer video files are inputted to the model, as the proposed system only uses frame extraction and frame-level classification.

Further enhancements of the proposed system are possible through the integration of NLP methods for analyzing textual information such as captions, comments, or descriptions. In this way, the proposed system will be able to conduct a more thorough analysis that utilizes both visual and textual data. Real-time processing, in which the system will have the ability to analyze video stream in real time, could be another enhancement. Such an update of the system will provide better applicability for video surveillance and other applications. In addition, this system can be enhanced to achieve high efficiency through optimization and can be deployed on the cloud to increase scalability and accommodate more users.

Therefore, with these enhancements, this system will be able to detect and prevent harmful content much more efficiently and effectively.

Appendix A

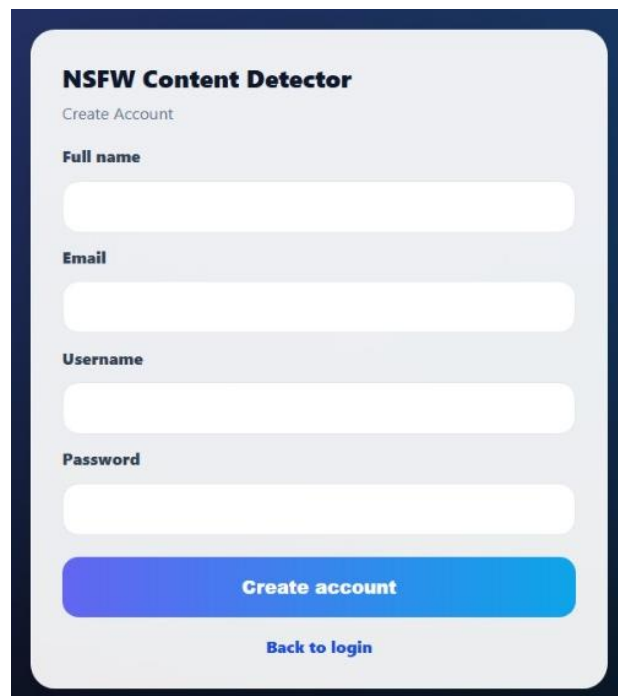
User Manual

System Requirements

For effective utilization of the system, the user should possess a computer either in the form of a desktop or a laptop that has a reliable internet connection along with a recent browser, for instance, Google Chrome or Microsoft Edge. Such requirements facilitate the system to run efficiently with no interruptions.

User Registration

The process of logging into the system involves registration. This requires a person to launch the web application and clicking on the “Register” button. Thereafter, the user is required to fill in their personal information including the first name, surname, e-mail address, password, confirm password, mobile number, and gender. Once all these sections have been filled successfully, one can now press the “Register” button, thereby allowing the user to log into the system, as shown in [Figure 1- A](#). User Registration:



The image shows a user registration form titled "NSFW Content Detector". The form is set against a light gray background with a dark blue border. It includes a "Create Account" link at the top. Below this, there are four input fields labeled "Full name", "Email", "Username", and "Password". At the bottom of the form, there is a prominent blue "Create account" button and a smaller, lighter blue "Back to login" link.

Figure 1- A. User Registration

User Login

After registering, the user may access the dashboard of the system using the registered email and password. Upon successful entry of the credentials and clicking of the “Login” button, access to the dashboard is granted, and failure to provide the correct credentials displays an error message, as shown in [Figure 2- A](#). User Login:

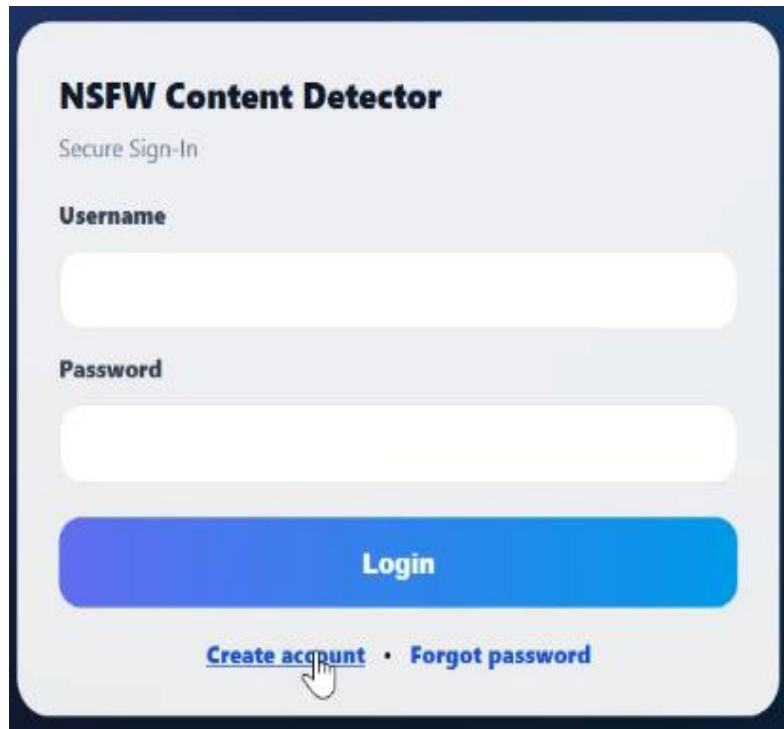
The image shows a login form titled "NSFW Content Detector" with the subtitle "Secure Sign-In". It features two input fields: "Username" and "Password". Below these fields is a prominent blue "Login" button. At the bottom of the form, there are two links: "Create account" (underlined) and "Forgot password". A mouse cursor is shown clicking on the "Create account" link.

Figure 2- A. User Login

Dashboard Overview

The user is then taken to the dashboard upon successful login. The dashboard is essentially the interface of the system whereby the user uploads his/her files and can analyze results generated earlier by the software. The dashboard makes it easy for the user to interact with the system, as shown in [Figure 3- A](#). Dashboard Overview:

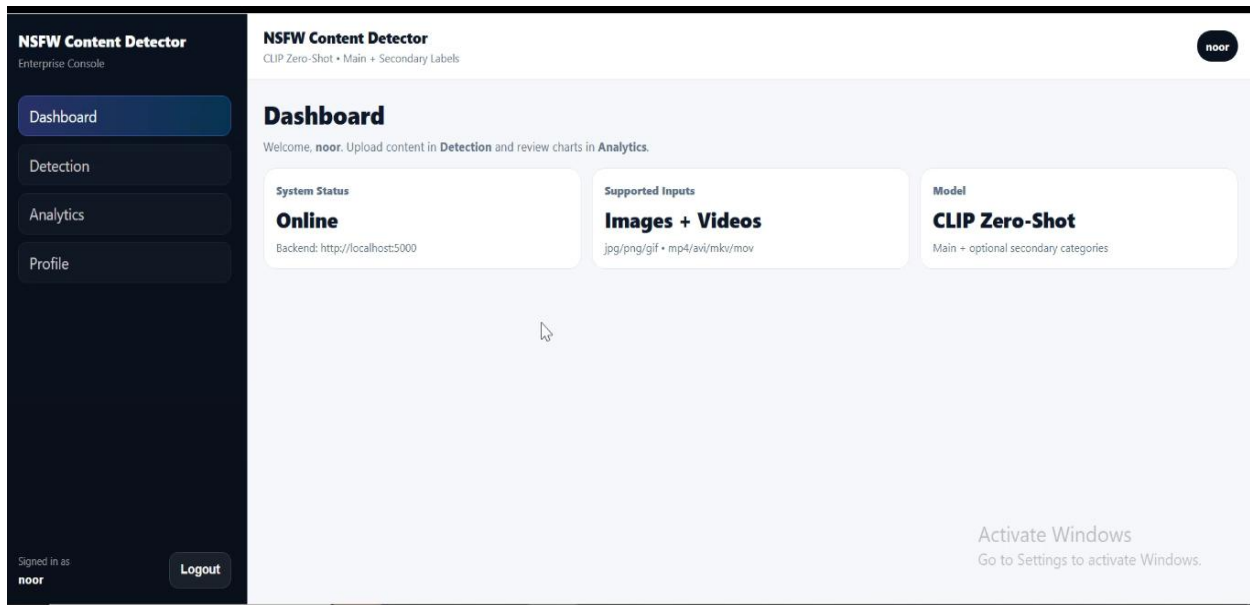


Figure 3- A. Dashboard Overview

Media Upload

To analyze content, the user must upload an image or video file. This is done by clicking on the “Upload” button and selecting a file from the device. The system supports common formats such as JPG and PNG and GIFs for images and MP4, AVI, MKV, MOV for videos. Once the file is selected, the user clicks on the “Analyze” button to upload it to the system, as shown in [Figure 4-A. Media Upload](#):

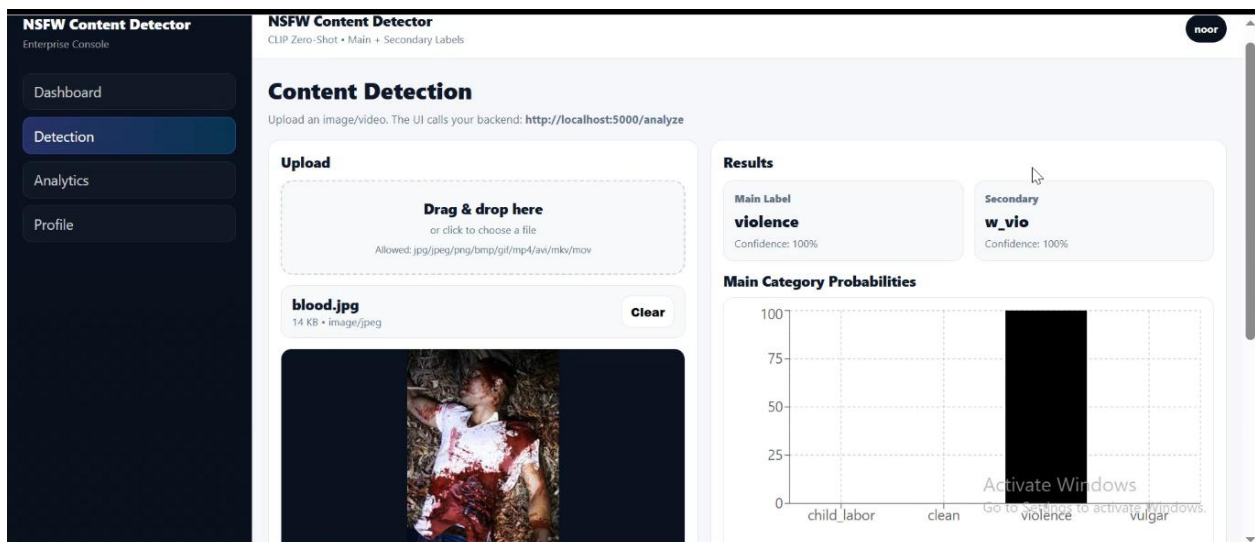


Figure 4- A. Media Upload

Output and Results

After the analysis process is completed, the outcome of the process is displayed in the dashboard. The output will consist of the classification category together with the probability level of the prediction made by the machine learning model. This output data is then saved into the database for future retrieval purposes. As shown in [Figure 5- A. Output and Results](#):

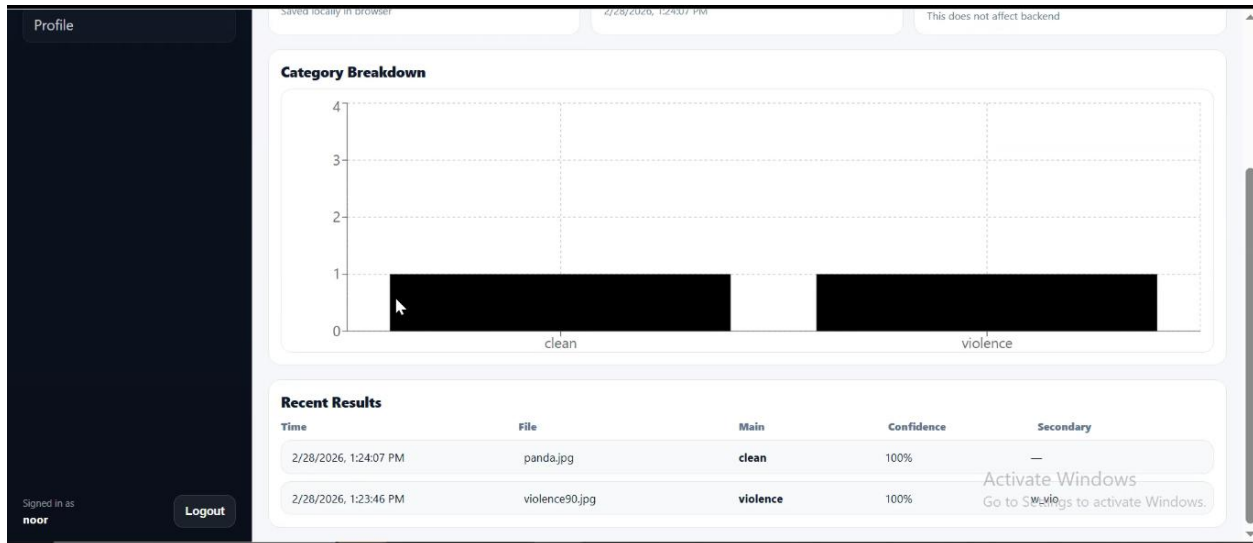


Figure 5- A. Output and Results

Error Handling

The system will have all the necessary error handling features to allow for efficient functioning. In case of incorrect login credentials being entered by the user, in the case of wrong file formats being uploaded, or the case where no file is chosen before submission, there will be suitable error notifications.

Logout

Once the user is done with the system, they have an option to log out by clicking on the “Logout” button. Once they click on it, their session will be terminated and redirected to the login screen.

References

- [1] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *International Conference on Learning Representations (ICLR)*, 2015. Cited on p. [4](#)
- [2] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning Transferable Visual Models from Natural Language Supervision," *arXiv preprint arXiv:2103.00020*, 2021. Cited on p. [4](#)
- [3] A. Howard, M. Sandler, G. Chu, L. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. Cited on p. [4](#) and [5](#)
- [4] N. Haider, A. Saleem, M. A. Khan, J. Jamal, and I. Ud Din, "Detection of Violent Activities in Videos Using Deep Learning," *IEEE Access*, vol. 8, 2020. Cited on p. [5](#)
- [5] M. Ali et al. NSFW content detection using deep neural networks. *Journal of Visual Communication*, 2021. Cited on p. [5](#)
- [6] International Labour Organization, "Global Estimates of Child Labour" Results and Trends, 2012–2016," ILO, Geneva, 2020. Cited on p. [5](#)
- [7] Yahoo Open NSFW Model (2016), Open NSFW Model by Yahoo Research. Cited on p. [5](#)
- [8] Skin Sheriff Project. *Skin-based Nudity Detection*. 2018. Achieved 91.9% accuracy using skin detection techniques for classifying nudity in images. Cited on p. [5](#)
- [9] TensorFlow. Open-source machine learning framework by Google.[5]
- [10] OpenCV Team, "OpenCV: Open-Source Computer Vision Library," 2023.
- [11] Flask Development Team, "Flask: A Lightweight WSGI Web Application Framework," Pallets Projects, 2023.
- [12] Meta Open Source, "React: A JavaScript Library for Building User Interfaces," 2023.